In The Name of God

Forecasting Competitive Football

Pre-Match and In-Play Prediction from Raw Event Data

Contributors: Parsa Bordbar (40435340), Navid Zare (40435071)

Data sources: StatsBomb Open Data; Football-Data.co.uk

Paper P1: Geometric SMOTE for Imbalanced Datasets with Nominal and Continuous Features

Paper P2: Hierarchical Shrinkage: Improving the Accuracy and Interpretability of Tree-Based Models

Dataset at a glance

Quantity	Value	Quantity	Value
Matches in store	54,983	Pre-match design columns	157
Modelled matches	51,459	In-play design columns	394
Seasons	2008/09 - 2025/26	Pre-match train / val / test	39,707 / 5,944 / 5,808
Leagues (train / test)	11 / 9	In-play train / test rows	17,024 / 6,536
In-play snapshots	28,823	Snapshots per match	19 (t = 0, 5, ..., 90)
Odds tagging (test)	5,806 / 5,808 = 99.97%	Best model gap to market	+0.00499 RPS

Contents

1. Introduction and Problem Framing

Football is the most widely followed sport in the world, and the analytics that support it rest on two very different kinds of record. The first is the match record: who played whom, where, when, and how the fixture finished. The second is event data, a complete time-stamped log of every action on the pitch, of which a single match yields roughly four thousand rows. The first kind is abundant and shallow; the second is scarce and deep. This project uses both, and the division of labor between them shapes every design decision that follows.

The task is to forecast the outcome of a football match at two distinct moments. Before kick-off, the only admissible evidence is what was knowable when the team sheets were published: recent form, long-run team strength, squad quality, venue, rest and head-to-head history. During the match, the evidence grows continuously, and a forecast made at minute t may use every event with a timestamp at or before t and nothing after it. These two regimes require separate feature pipelines with separate leakage barriers, and the brief is explicit that constructing them honestly is the heart of the project.

A forecast is only meaningful relative to reference points. Two are used throughout. The floor is a prior-rate dummy that ignores every feature and predicts the training-set class distribution; any model that fails to clear it is broken. The ceiling is the de-vigged Bet365 1X2 price, which aggregates the beliefs of a market with money at risk and is, for practical purposes, the best publicly available probability estimate for a football match. Beating that ceiling is a very high bar, and a carefully analyzed failure to beat it is a more valuable result than an unexplained score that merely looks good.

1.1 Formal definition of the three tasks

Model 1 - Pre-match outcome classification (Task C)

One example is one match. The feature vector describes only quantities computable strictly before kick-off. The label is the full-time result encoded on the ordered scale H (home win), D (draw), A (away win). The output is a probability vector over those three classes summing to one. The training set is every match whose season falls at or before the pinned training boundary and which is not reserved as an in-play holdout, giving 39,707 examples; the test set is the 5,808 matches of the two most recent complete seasons.

Model 2 - Pre-match goal-margin regression (Task R)

One example is one match, described by the identical pre-match feature vector. The label is the signed goal margin, home goals minus away goals, clipped to the interval [-5, +5] so that a single rout cannot dominate the squared error. The output is a real number on that interval. Clipping is applied to both the training label and the prediction, so the model is never asked to reproduce a value it is not permitted to emit.

$$y_r = \text{clip}\ (\text{home\_goals} - \text{away\_goals}, -5, +5)$$

Model 3 - In-play prediction (Tasks Lc and Lr)

One example is one match observed at one moment t. For every match in the event-data subset, nineteen snapshots are cut at t = 0, 5, 10, ..., 90 minutes. The feature vector concatenates the pre-match vector for that fixture with an in-play block computed exclusively from the prefix of the event stream whose effective minute is at most t. The label is the same full-time result (Task Lc) or the same clipped margin (Task Lr) as the pre-match tasks: the target never changes, only the

information available to predict it. Nineteen snapshots per match over 1,517 matches give 28,823 rows, of which 17,024 train and 6,536 test.

The critical consequence of this definition is that snapshots from a single match are not independent. Nineteen rows share one label and a large common prefix of evidence. Every split, every resampling decision and every significance test in this report therefore operates on match identifiers, never on rows.

1.2 Evaluation metrics and why each was chosen

The three outcomes lie on a natural ordinal scale: if the true result is a home win, forecasting a draw is a smaller error than forecasting an away win. The Ranked Probability Score is the proper scoring rule that respects that ordering, and it is the headline metric for both classification heads. With r = 3 ordered categories, forecast probabilities $p_j$and one-hot outcome indicators e_j, it compares cumulative distributions:

$$\boldsymbol{RPS = \frac{1}{r-1}\sum_{i=1}^{r-1}\left(\sum_{j=1}^{i}(p_j - e_j)\right)^2}$$

Lower is better and zero is a perfect confident forecast. Two order-blind scoring rules are reported alongside it. Log-loss punishes confident mistakes severely, diverging as a realised outcome is assigned probability zero, and the multiclass Brier score applies a gentler squared error to the whole probability vector:

$$\text{LogLoss} = -\frac{1}{N}\sum_{n}\sum_{k} o_{nk}\ln(p_{nk})$$

$$\text{Brier} = \frac{1}{N}\sum_{n}\sum_{k}(p_{nk} - o_{nk})^2$$

Calibration is measured by the Expected Calibration Error over ten equal-width confidence bins, where acc(B) is the accuracy of the predictions falling in bin B and conf(B) their mean confidence:

$$ECE = \sum_{m=1}^{M}\frac{|B_m|}{N}|\text{acc}(B_m) - \text{conf}(B_m)|$$

For the regression heads the report gives mean absolute error, which is the average size of the miss in goals; root mean squared error, which is dominated by the occasional heavy defeat that was not foreseen; and the correlation between predicted and realized margins, which separates ranking skill from scale calibration. All probability vectors are floored at 0.005 per class and renormalized before scoring, so that a single confidently wrong forecast cannot make log-loss infinite.

1.3 Research questions

- **RQ1.** How close can an odds-free model trained on public match data come to the de-vigged bookmaker price, and does the gap close as the training sample grows?
- **RQ2.** At what point during a match does live evidence overtake a frozen pre-match forecast, and which in-play features carry that gain?

- **RQ3.** Does synthetic minority oversampling, and specifically the G-SMOTENC procedure of paper P1, improve the treatment of the draw class once the training sample is large?
- **RQ4.** Does hierarchical shrinkage, paper P2, improve a random forest on tabular football data relative to modern gradient boosting?
- **RQ5.** Does ensembling by stacking beat the strongest single learner on any of the four prediction heads, and can that claim be supported statistically?
- **RQ6.** Do kernel methods scale as their theoretical complexity predicts on a design matrix of this size?

Each question is answered explicitly in Section 9.2, and each answer is tied to a numbered table or figure in the body of the report.

2. Dataset, Integration Design and Competition Selection

The store behind this report is assembled from four public sources that do not share a single key between them. Reconciling them is the first graded deliverable and consumed the largest share of the engineering effort.

2.1 Sources, provenance and download links

Every raw input is fetched by a single reproducible script, src/pipeline/download_extended_data.py, which downloads each file, verifies its shape, and records a truncated SHA-256 digest in download_manifest.csv. Ninety-three files are tracked. The sources and their canonical addresses are:

Table 1. *The four public sources behind the store, what each contributes and where it is retrieved from. Every address is the exact one used by download_extended_data.py.*

Source	What it contributes	Address
European Soccer Database	Match results, lineups and historical odds for 25,979 fixtures across 11 leagues, seasons 2008/09-2015/16	huggingface.co/datasets/julien-c/kaggle-hugomathien-soccer/resolve/main/database.sqlite
Football-Data.co.uk	Season CSVs carrying results and bookmaker prices, seasons 2016/17-2025/26, divisions E0, SP1, I1, F1, D1, N1, P1, B1, SC0	Pattern: www.football-data.co.uk/mmz4281/{season}/{div}.csv - example: www.football-data.co.uk/mmz4281/2425/E0.csv
StatsBomb Open Data	Event streams, lineups and 360 freeze frames; the source of every in-play feature	github.com/statsbomb/open-data
sofifa ratings, legacy archive	Player overall, potential, age and position for FIFA 15 to FIFA 23	huggingface.co/datasets/jsulz/FIFA23/resolve/main/male_players (legacy).csv
sofifa ratings, all updates	Every rating update, 5.6 GB upstream, streamed and filtered in flight to the eleven modelled leagues	huggingface.co/datasets/jsulz/FIFA23/resolve/main/male_players.csv

The division of labor is deliberate. The European Soccer Database and the Football-Data season files supply breadth: 54,983 fixtures across eleven leagues and eighteen seasons, enough to train a pre-match model that is not sample-starved. StatsBomb supplies depth: complete event streams for the four 2015/16 big-five league seasons, without which no in-play model is possible at all. The sofifa archives supply squad quality, which neither of the other two records. Bet365 1X2 prices, carried by both the Football-Data CSVs and the European Soccer Database odds columns, supply the market ceiling.

2.2 Competition selection

Two selection decisions were made, one for each pipeline, and both are defensible on the same principle: a competition enters the study only if it contributes a full league season with an attached odds source, because tournaments and women's competitions add distributional variety without adding history or a market baseline.

For the in-play pipeline the audit ran over all eighty StatsBomb competition-seasons. Table 2 reports the four that survived and Table 3 the grounds on which the rest were rejected. The four retained seasons are the only ones in the archive that are simultaneously complete league seasons, men's club football, and covered by a Football-Data division file.

Table 2. *Competition-seasons retained for the in-play (event-data) pipeline. The top-team share confirms a balanced double round-robin in every case.*

Competition	Season	Matches	Teams	Top-team share	First	Last
La Liga	2015/2016	380	20	0.100	2015-08-21	2016-05-15
Premier League	2015/2016	380	20	0.100	2015-08-08	2016-05-17
Serie A	2015/2016	380	20	0.100	2015-08-22	2016-05-15
Ligue 1	2015/2016	377	20	0.101	2015-08-07	2016-05-14
Total		**1517**	**80**			

Table 3. *Grounds for rejecting the remaining StatsBomb competition-seasons. No rejected entry has an attached 1X2 odds source, so none could be scored against the market ceiling.*

Reason for exclusion	Competition-seasons	Matches
league not in odds source	1	115
single-club / finals	56	705
tournament (no odds source)	6	314
women's (no odds source)	13	1310
Total rejected	**76**	**2444**

For the pre-match pipeline the selection is inherited from the sources. The European Soccer Database covers eleven leagues; the nine Football-Data divisions cover nine of them. Ekstraklasa and the Swiss Super League therefore exist only in the 2008-2016 portion of the store. Table 4 makes the consequence explicit and it is important enough to state in plain terms: the training block spans eleven leagues while validation and test span nine. Those two extra competitions contribute 3,342 training matches with no odds attached at all, which is why store-wide odds coverage is 93.5 per cent while test-block coverage is 99.97 per cent. This is a measured distribution shift, not a hidden one, and Section 3.4 returns to what it costs.

Table 4. *Odds coverage by league over every season inside the pinned split window. The two zero-coverage leagues are present only in the 2008-2016 era and therefore appear in the training block alone.*

League	Matches	Tagged with odds	Coverage	Appears in
Bundesliga	5202	5201	0.9998	train / val / test
Ekstraklasa	1920	0	0.0000	train only
Eredivisie	5128	5124	0.9992	train / val / test
La Liga	6460	6459	0.9998	train / val / test
Ligue 1	6208	6202	0.9990	train / val / test
Premier League	6460	6460	1.0000	train / val / test
Primeira Liga	4806	4797	0.9981	train / val / test
Pro League	4210	4197	0.9969	train / val / test
Scottish Premiership	3827	3825	0.9995	train / val / test
Serie A	6437	6431	0.9991	train / val / test
Swiss Super League	1422	0	0.0000	train only
All modelled seasons	**52080**	**48696**	**0.9350**	

2.3 Relational integration and identity resolution

The three match sources name the same clubs differently and key them differently. The European Soccer Database uses integer team API identifiers, Football-Data uses free-text club names that vary by season, StatsBomb uses its own team identifiers, and the sofifa archives use club names again with yet another spelling convention. A canonical team registry was therefore built as the spine of the store: 358 canonical clubs, each carrying the league it belongs to and the source from which its identifier was minted. An alias map resolves every source-specific name, per league and per season, onto a canonical identifier, and the odds join is validated one-to-one so that a duplicated or unresolved alias fails the build rather than silently dropping rows.

Match identity is resolved on the tuple (league, season, match date, home canonical id, away canonical id). This is stricter than joining on date and team names, and it is what allows the Football-Data odds file and the European Soccer Database results table to be merged without any fuzzy string comparison at match level. The StatsBomb matches are attached by their own match identifier and tagged with era = statsbomb, so that the three provenances remain distinguishable downstream.

2.4 Odds integration and de-vigging

Bookmaker prices are quoted as decimal odds, which are reciprocals of implied probabilities inflated by the bookmaker's margin. The three reciprocals sum to an overround strictly greater than one. The market baseline is recovered by the standard multiplicative normalisation, applied per match:

$$p_k = \frac{1/O_k}{\sum_j 1/O_j}, \quad k, j \in \{H, D, A\}$$

The build asserts that every resulting triple sums to one within 1e-9 and that every overround exceeds one, so a corrupted or partially missing price row cannot enter the baseline. Bet365 is the preferred source everywhere. Where the European Soccer Database lacks a complete Bet365 triple for a fixture, the median across the ten bookmakers present in that table is substituted and the row is flagged as bookie_median; where no complete triple from any bookmaker exists, the fixture is excluded from the market comparison and written to an explicit failure log rather than dropped. A hard assertion requires at least 98 per cent odds coverage of the test block before the stage will write its output.

Table 5. *Odds tagging coverage of the test block by competition. Only two Pro League fixtures out of 5,808 could not be tagged; both are logged with their reason and excluded from the market comparison rather than imputed.*

Competition	Test matches	Tagged with odds	Coverage
Bundesliga	612	612	1.0000
Eredivisie	612	612	1.0000
La Liga	760	760	1.0000
Ligue 1	612	612	1.0000
Premier League	760	760	1.0000
Primeira Liga	612	612	1.0000
Pro League	624	622	0.9968
Scottish Premiership	456	456	1.0000
Serie A	760	760	1.0000
All test leagues	**5808**	**5806**	**0.9997**

2.5 Data-quality log

Nothing is silently repaired. Every corrective action taken during the build is appended to a data-quality log with its scope, the field affected, the number of rows involved and the reason. The dominant entry concerns the StatsBomb event streams: timestamps in the raw JSON are not monotone, because the recorded minute occasionally regresses within a period. Sorting on the raw minute would therefore produce a non-causal ordering and would corrupt every in-play prefix. Events are instead ordered canonically on (period, index) and an effective minute is derived as a running maximum:

$$\text{eff\_minute}(i) = \max_{j \le i} \text{minute}(j), \quad \text{over events sorted by (period, index)}$$

This guarantees a non-decreasing time axis and makes the snapshot cut a true prefix of the stream. The log records 1,526 entries in total. Separately, cleaning_drops.csv is empty for this run: no record was discarded by the cleaning stage, so every exclusion in this report is a deliberate split-assignment decision rather than a data loss.

2.6 Temporal splits

The split is by season boundary and its shape is fixed by contract: the two most recent complete seasons are the test block, the two before them are validation, and everything earlier trains. The four constants are pinned in source rather than derived from whatever seasons happen to be on disk, because downloading one further season must not silently reshape the split underneath a finished set of results. Assertions enforce that the maximum training date precedes the minimum validation date and the maximum validation date precedes the minimum test date, so the contract 'predict forward, never backward' cannot be violated by a mis-specified constant.

Two buckets sit outside the four standard ones. The 2025/26 season is present in the store but assigned out_of_scope and appears nowhere downstream; it is loaded so that rolling the window forward is a four-line change rather than a re-download. The 621 StatsBomb matches that form the in-play validation and test blocks are assigned excluded and removed from pre-match training, so that the frozen pre-match reference used in Section 8B is never scored on a fixture the pre-match model was fitted on.

Table 6. *Chronological split of the 54,983-match store. Validation and test each span two complete seasons; the excluded bucket is the StatsBomb in-play holdout and the out-of-scope bucket is the loaded but unused 2025/26 season.*

Split	Matches	Seasons	First match	Last match	Leagues
train	39,707	2008/2009 - 2020/2021	2008-07-18	2021-05-23	11
validation	5,944	2021/2022 - 2022/2023	2021-07-23	2023-06-04	9
test	5,808	2023/2024 - 2024/2025	2023-07-28	2025-05-25	9
excluded	621	2015/2016 - 2015/2016	2016-02-01	2016-05-17	4
out_of_scope	2,903	2025/2026 - 2025/2026	2025-07-25	2026-05-24	9
All	**54,983**	**2008/2009 - 2025/2026**	**2008-07-18**	**2026-05-24**	**11**

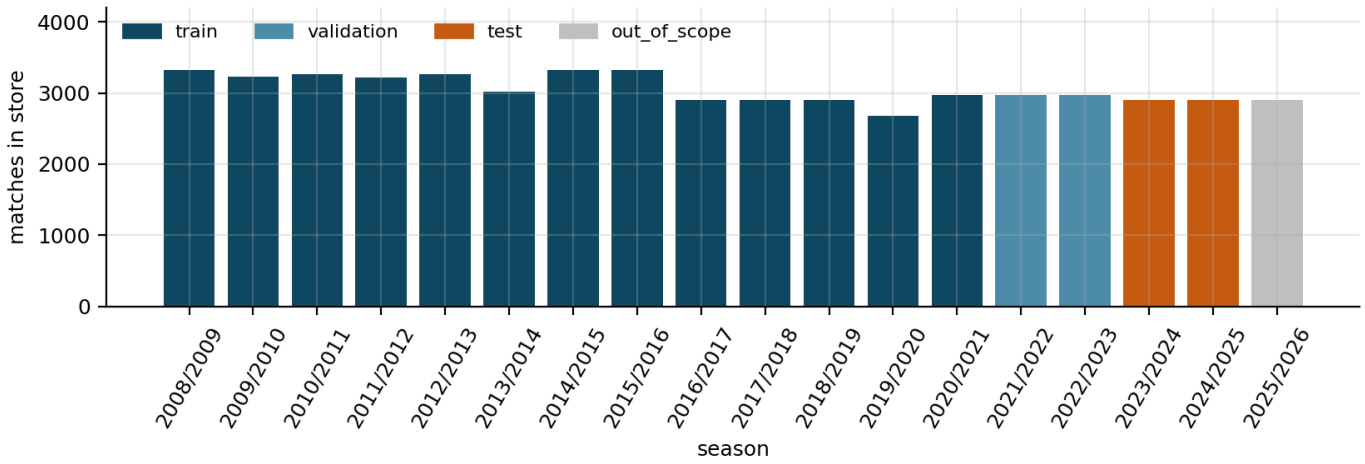

Figure 1. *Matches per season in the store, coloured by split assignment. The step at 2016/17 is the transition from the European Soccer Database to the Football-Data season files, which also removes two leagues; the 2025/26 bar is loaded but scores nothing.*

Figure 1 shows why the training block is not a homogeneous population. Volume per season rises from roughly 3,000 to roughly 3,300 fixtures at the source transition and then holds steady, but the number of leagues drops from eleven to nine at the same point. The expected consequence, confirmed in Section 3.4, is that a small number of feature columns are populated in the earlier era and empty in the later one, and the design has to declare that rather than let the imputer hide it.

3. Feature Engineering: Pre-Match and In-Play Pipelines

Two pipelines are built from one store. They share a vocabulary of quantities but differ completely in what window of history each quantity is computed over, and in what constitutes a violation of causality.

3.1 The pre-match design matrix

The pre-match table carries 157 model columns per match, organized into six families. Rolling form averages goals for, goals against, points and wins over a team's five previous matches, computed on a per-team long table and shifted by one so that the current fixture never contributes to its own features. Venue form repeats those four quantities restricted to home or away fixtures respectively. Head-to-head averages goals and points over the three previous meetings of the same ordered pair. Schedule carries rest days since the previous fixture and the count of matches already played, which doubles as a maturity indicator for the rolling windows. Match-stat form aggregates the shots, shots on target, corners, fouls, yellow and red cards recorded in the Football-Data files over the same prior window. Every quantity is emitted three times: for the home team, for the away team, and as a signed home-minus-away difference, because tree learners split far more efficiently on the difference than on the pair.

Two rating layers sit on top. Elo is maintained per league with a home-field bonus, a season-boundary regression toward the league mean, and a penalty for newly promoted clubs that have no history. Its update is the textbook form, applied after the fixture is scored so that only the pre-match rating ever enters the design matrix:

$$E_h = \frac{1}{1 + 10^{-(R_h + H - R_a)/400}}$$

$$R_h' = R_h + K(S - E_h), \quad R_a' = R_a - K(S - E_h)$$

where S is 1, 0.5 or 0 for a home win, draw or away win. *Pi-ratings* maintain separate home and away strengths per club and are updated from the goal-margin error rather than the result, using a logarithmic learning function that damps large errors:

$$\hat{gd} = g\left(R_h^H\right) - g(R_a^A), \quad g(R) = \text{sign}(R) \cdot \left(10^{|R|/c} - 1\right)$$

$$\psi(e) = c \log_{10}(1 + |e|), \quad e = (h - a) - \hat{gd}$$

$$R_h^H \mathrel{+}= \lambda\psi, \quad R_h^A \mathrel{+}= \gamma\lambda\psi, \quad R_a^A \mathrel{-}= \lambda\psi, \quad R_a^H \mathrel{-}= \gamma\lambda\psi$$

The three free rating parameters were tuned by grid search against validation RPS before any learner was fitted, which keeps the rating layer honest: it is selected on validation data only and then frozen. Table 7 gives the top of that search. The selected configuration is K = 10, home advantage 90 Elo points and lambda = 0.06, and the search surface is notably flat - the best and worst of thirty-six configurations differ by 0.0004 RPS - which says the rating layer is robust to its own hyperparameters.

Table 7. Rating-layer grid search, best six of thirty-six configurations with the worst shown last for range. Scored by validation RPS with a plain logistic readout; the selected row is the first.

Elo K	Elo home advantage	Pi lambda	Validation RPS
10	90	0.060	0.198789
10	70	0.060	0.198795
10	50	0.060	0.198803
20	90	0.035	0.198871
20	70	0.035	0.198888
32	90	0.035	0.198930
32	**50**	**0.090**	**0.200001**

The final family is squad quality, derived from the sofifa player archives. Two constructions are used. Where real lineups exist, an XI aggregate summarizes the eleven named starters by line. Everywhere, a squad aggregate summarizes the club's roster - top-eleven and top-eighteen mean overall, best goalkeeper, per-line means, mean age - from the most recent rating snapshot strictly before the match date. Both joins are as-of joins with exact matches disallowed and a post-join assertion that no attached rating carries a date at or after the fixture.

3.2 The in-play snapshot design

The in-play table carries 394 model columns. Roughly 240 of them are the pre-match vector for the fixture, reused unchanged at every snapshot; the remainder are computed from the event prefix. The in-play block records the live score and goal difference, shots and shots on target, an expected-goals proxy from shot locations, man advantage from dismissals, event counts, possession share, passing volume by pitch third, carries and touches in the attacking third, pressure events, defensive actions, turnovers and set-piece counts. Each of these appears as a home value, an away value and a signed difference, and additionally in two derived forms: a rate per elapsed minute, and a recent form restricted to a trailing window, which is what carries momentum rather than accumulated volume.

Goal attribution deserves a note because it caused a real defect during development. In the StatsBomb schema an own goal is recorded twice, and the event type Own Goal For is attached to the benefiting team rather than to the player who scored it. Crediting the scoring team's tally, which is the intuitive reading, produced 126 matches whose reconstructed scoreline disagreed with the official result. The corrected rule counts, for a given team, its own scoring events plus the Own Goal For events recorded against its opponent, and the reconstruction now agrees with the match store on every fixture.

3.3 Leakage barriers

Four barriers are implemented, each with an assertion that fails the build rather than a comment that documents an intention.

Barrier one: chronological splitting

Splits are assigned by season, never at random. The build asserts that the training block ends strictly before validation begins and validation ends strictly before test begins. Random splitting on this data would place a match from March 2024 in training and a match from September 2023 in test, and every rating and form feature would then encode information from the future of the test fixture.

Barrier two: prior-match-only form

Every rolling quantity is computed with a shift of one position within the team's own ordered history before the window is applied. The build asserts that the first match of every team carries missing values for all rolling form columns and a played-prior count of zero. Both rating layers write only the pre-update value into the feature row and apply the update afterwards.

Barrier three: the effective-minute prefix cut

The snapshot at minute t uses the events whose effective minute is at most t and no others. The cut is found by binary search on the monotone effective minute array, and two assertions guard it: the last included event must have an effective minute at most t, and the first excluded event must have an effective minute strictly greater than t. Together these state that the cut is a genuine prefix, with no event straddling the boundary in either direction.

Barrier four: train-only transform fitting

The median imputer, the standardizer and the one-hot encoder are fitted on the training rows alone and then applied unchanged to validation, test and the frozen-reference rows. Fitting the imputer on the pooled table would leak the test-block median of every column into the training representation. Similarly, all resampling is fitted inside the training block only and is forbidden outright on the snapshot table, because oversampling across matches would synthesise a row interpolating two different fixtures.

3.4 Known limitations of the feature pipelines

Three limitations are structural rather than incidental, and all three are visible in the ablation results of Section 8E. They are stated here so that later numbers are read correctly.

The XI-rating family is empty on the test block. Real starting elevens are available only from the European Soccer Database lineup columns, which stop in 2016. Every test fixture belongs to the Football-Data era, so the indicator column xi_available is zero throughout validation and test and all twenty-seven XI columns are missing, to be filled by the training-block median. These columns are therefore constant on the evaluation data and carry no signal there. The ablation confirms the diagnosis rather than contradicting it: removing the XI family improves validation RPS on Task C, from 0.19771 to 0.19769, and improves Task Lr mean absolute error by 0.0035. They are retained in the reported design matrix for continuity with the in-play pipeline, where lineups do exist, but a production pre-match model should drop them.

Squad ratings are frozen at FIFA 23 across the whole test window. The optional FC 24 and FC 25 roster files are not public in a form the download script can verify, so the most recent available snapshot for a 2024/25 fixture is a rating published in 2023. Since diff_squad_overall_top18 and diff_squad_overall_top11 are the fourth and fifth most important features on Task C by mean absolute SHAP, this staleness is a genuine ceiling on pre-match performance and a clear first target for future work.

Two training leagues are absent from the evaluation blocks. As Table 4 showed, Ekstraklasa and the Swiss Super League contribute 3,342 training matches from competitions that never appear in validation or test. The league identity is an explicit one-hot feature and the era of provenance is another, so the learner can in principle partition on them; SHAP shows it essentially does not, with every league indicator and both era indicators scoring below 0.0012 mean absolute SHAP on Task C. The shift is therefore measured and found to be small, which is the outcome the brief asks for, but it is not zero and it should not be described as absent.

4. Paper Reproductions (P1 and P2)

Two papers were reimplemented from the published description, without consulting any reference implementation. Both had to satisfy the same hard filter before selection: published in 2022 or later, in a Q1 Scimago venue or a CORE A/A* conference, non-deep-learning, applicable to the data at hand, and reducible to a core equation that can be derived by hand. Table 8 records the audit.

Table 8. *Hard-filter audit for both selected papers. Every criterion is satisfied by both, and the venue tier is the binding constraint that eliminated several earlier candidates.*

Criterion	P1 - G-SMOTENC	Verdict	P2 - Hierarchical Shrinkage	Verdict
Publication year 2022 or later	2023	Pass	2022	Pass
Venue quality	Expert Systems with Applications	Q1 Scimago	ICML (PMLR 162)	CORE A*
Non-deep-learning	Geometric oversampling; no network	Pass	Post-hoc tree regularization; no network	Pass
Applicable to this data	Nominal and continuous mixed features	Pass	Any tree ensemble, both task types	Pass
Reimplementable from scratch	Full algorithm in the paper	Pass	Single recursion in the paper	Pass
Hand-derivable core equation	Hyperball surface sampling	Pass	Telescoping path sum (Appendix A)	Pass

4.1 P1 - Geometric SMOTE for nominal and continuous features

Fonseca and Bacao (2023) extend Geometric SMOTE to design matrices that mix nominal and continuous columns. The motivation for using it here is direct: the draw is the minority class in every football data set, occupying 25.6 per cent of the 5,808 test fixtures against 43.2 per cent for home wins, and every learner in the zoo predicts it almost never.

The algorithm has three parts. For a chosen minority point, a partner is selected either from the minority class, from the majority class, or from whichever is nearer under a combined strategy. A synthetic point is then drawn on the surface of a hyperball centred on the minority point, rather than on the straight segment between the pair as ordinary SMOTE does. The radius is the distance to the partner; the drawn direction is truncated toward the partner by a truncation factor in [-1, 1] and deformed toward the minority-partner axis by a deformation factor in [0, 1]. Sampling from a surface rather than a chord is what gives the method its name and is the part that must be derived rather than copied: a direction is drawn uniformly on the unit sphere, the component parallel to the minority-partner axis is measured, and the perpendicular component is scaled down by the deformation factor before the radius is applied.

The nominal columns cannot be interpolated, so the paper's third contribution is a distance and an assignment rule for them. Nominal columns are encoded for the neighbor search with a weight tied to the median standard deviation of the continuous block, so that a category mismatch costs a comparable distance to a one-deviation continuous gap; a synthetic point then inherits each nominal value from whichever of its two parents is nearer, rather than receiving a meaningless average. The implementation exposes categorical_features, k_neighbors, selection_strategy, truncation_factor and deformation_factor exactly as the paper defines them, asserts each is in its admissible range,

and is covered by unit tests that check the drawn points lie within the stated radius, that nominal values are always one of the two parents' values, and that the resampled class counts are balanced.

P1 is a training-time toggle only. It is fitted inside the training block and never written into a feature file, never applied to validation or test, and explicitly forbidden on the snapshot table. Its measured effect is reported in Section 8C and it is not favorable.

4.2 P2 - Hierarchical Shrinkage

Agarwal, Tan, Ronen, Singh and Yu (ICML 2022) propose a post-hoc regularization of any fitted tree ensemble. The prediction of a tree is rewritten as a telescoping sum along the root-to-leaf path, and each increment is shrunk toward its parent by an amount that depends on how few samples the parent held:

$$f_{HS}(x) = \mu(t_0) + \sum_{l=1}^{L} \frac{\mu(t_l) - \mu(t_{l-1})}{1 + \lambda / N(t_{l-1})}$$

Here t_0 is the root, t_L the leaf reached by x, mu(t) the mean response in node t and N(t) its sample count. The shrinkage is strongest exactly where the data is thinnest, which is the deep end of the tree, and it vanishes as N grows. The derivation of this identity from the recursive definition is given in Appendix A. The property that makes it attractive is that it touches only the node values: the tree structure, the split variables and the split thresholds are all unchanged, so shrinkage costs nothing at prediction time and one fitted forest can be re-shrunk at many values of lambda without refitting.

The implementation shrinks a fitted scikit-learn forest in a single downward pass over each tree, reading weighted node sample counts and node means directly from the tree structure. Lambda is selected from the grid {0, 0.1, 1, 10, 25, 50, 100} by internal cross-validation on the training block alone, never on validation or test. Table 9 records the fidelity audit.

Table 9. *P2 fidelity audit: the property claimed by the paper, how it is realized in the reimplementation, and the test that holds it.*

Property claimed	Implementation	Verification
Post-hoc, structure preserved	Node values overwritten; children_left, feature and threshold untouched	Unit test compares tree topology before and after
Telescoping path recursion	Iterative stack from root, shrunk[child] = shrunk[parent] + (mu[child] - mu[parent]) * w	Closed-form check against a hand-computed three-node tree
Shrinkage weight	w = 1 / (1 + lambda / N(parent))	Asserted in [0, 1]; equals 1 at lambda = 0
lambda = 0 is the identity	Selection grid includes 0	Unit test asserts predictions identical to the unshrunk forest
Classification and regression	Separate classifier and regressor wrappers over the same shrink routine	Both exercised on all four heads
Interpretability claim	Shrunk values materialised back into the sklearn tree before TreeSHAP	Without this step TreeSHAP silently explains the unshrunk forest

The final row is the subtlest and was found by inspection rather than by a failing test. TreeSHAP walks the fitted tree object rather than the wrapper's predict method, so unless the shrunk values are written back into the tree's value array, the attributions returned describe the unshrunk forest and silently contradict the model actually being served. The wrapper therefore exposes a dedicated SHAP base estimator that carries the shrunk values.

P2 is a full member of the model zoo and competes on all four heads. Its results are honest but unspectacular: it is the second-best Task C model at RPS 0.19461, second-best on Task Lc at 0.14841, and fourth on the two regression heads. It never wins a head, and Section 6 shows that no tree-versus-tree difference on the pre-match tasks is statistically separable in any case. The paper's claim is that shrinkage helps most when trees are deep and leaves are thin; with 39,707 training rows and a tuned maximum depth of eight, the leaves are not thin, and the mechanism has little to bite on. That is a fair test of the paper rather than a failure of the reimplementation, and it is reported as such.

5. Experimental Setup

5.1 The model zoo

Six families are compared so that the question 'does the model class matter?' can be answered with evidence rather than assumption. Every family is non-deep-learning, as the brief requires.

Table 10. *The model zoo: every learner, the family it represents, the tasks it covers and its role in the comparison.*

Model	Family	Tasks	Role
dummy	Prior rate	C, R, Lc, Lr	Sanity floor; ignores every feature
random_forest	Bagging	C, R, Lc, Lr	Low-tuning variance reference
gbm	Boosting (histogram)	C, R, Lc, Lr	scikit-learn HistGradientBoosting
xgboost	Boosting	C, R, Lc, Lr	Served pre-match classifier
lightgbm	Boosting	C, R, Lc, Lr	Leaf-wise boosting contrast
p2_hier_shrinkage	Bagging + P2	C, R, Lc, Lr	Paper reimplementation (Section 4.2)
kernel_svm	Kernel	C	RBF support-vector classifier
kernel_svr	Kernel	R	RBF support-vector regressor
kernel_ridge_exact	Kernel	R	Exact Gram solve, capped at 8,000 rows
kernel_ridge_nystroem	Kernel	R	Nystroem approximation, 1,000 components
stack	Ensemble	C, R, Lc, Lr	Logistic / linear meta-learner on out-of-fold member outputs
stack_temporal	Ensemble	C, R, Lc, Lr	Same members, meta-learner fitted on early validation

The two ensembles differ only in how the meta-learner is trained. stack is textbook stacking: five-fold out-of-fold predictions over the training block, so the meta-learner never sees a member's in-sample output. stack_temporal fits the members once on the whole training block and fits the meta-learner on the earliest sixty per cent of the validation block. The design argument is that random folds spanning 2008 to 2021 do not transfer to a 2023-25 test block, whereas recent validation data does. Section 6.6 tests that argument and, for the first time in this project, finds statistical support for it. Kernel members are excluded from both stacks on cost grounds, for reasons Section 8D quantifies.

5.2 Hyperparameter search protocol

Each tunable model receives twelve candidate configurations drawn without replacement from its declared grid using a model-and-task-specific seed, and each candidate is scored by three-fold cross-validation on the training block. The objective is log-loss for classification and mean absolute error for regression. For the two in-play heads the folds are GroupKFold on match identifier, so that nineteen snapshots of one match can never be split across a fold boundary; for the pre-match heads, where a row is already a match, stratified and plain K-fold are used respectively.

Two budget decisions are recorded rather than hidden. First, the search runs on at most 8,000 training rows; above that threshold the block is subsampled, and for the grouped case the subsample is drawn at match level so the grouping survives. Every row of tuning_results.csv carries a subsampled flag recording this. The selected configurations are therefore optimised for a smaller training set than the one they are finally fitted on, which most plausibly leaves the boosted models slightly under-capacity. Second, the search was not re-executed in the final pass; the stored configurations were reused. This is sound because the model rows produced with and without the re-execution are bit-identical, so the parameters match the data they are applied to, but it is stated rather than implied.

Table 11. *Selected hyperparameter configuration for every tuned model and task, as written to best_params.json and consumed by every downstream stage.*

Task	Model	Selected configuration
C	random_forest	max_depth=8, max_features=sqrt, min_samples_leaf=10, n_estimators=500
C	gbm	l2_regularization=1.0, learning_rate=0.02, max_iter=200, max_leaf_nodes=15, min_samples_leaf=10
C	xgboost	colsample_bytree=0.6, learning_rate=0.02, max_depth=3, n_estimators=300, reg_lambda=1.0, subsample=0.8
C	lightgbm	colsample_bytree=0.6, learning_rate=0.02, n_estimators=400, num_leaves=15, reg_lambda=5.0, subsample=0.6
C	p2_hier_shrinkage	max_depth=8, n_estimators=500
C	kernel_svm	C=0.5, gamma=scale
R	random_forest	max_depth=8, max_features=0.3, min_samples_leaf=1, n_estimators=300
R	gbm	l2_regularization=1.0, learning_rate=0.02, max_iter=200, max_leaf_nodes=15, min_samples_leaf=10
R	xgboost	colsample_bytree=0.6, learning_rate=0.02, max_depth=3, n_estimators=200, reg_lambda=5.0, subsample=0.6
R	lightgbm	colsample_bytree=0.8, learning_rate=0.02, n_estimators=200, num_leaves=15, reg_lambda=5.0, subsample=0.8
R	p2_hier_shrinkage	max_depth=8, n_estimators=200
R	kernel_svr	C=0.1, epsilon=0.1, gamma=scale
R	kernel_ridge_exact	alpha=10.0, gamma=None
R	kernel_ridge_nystroem	alpha=10.0, gamma=None, n_components=1000
Lc	random_forest	max_depth=8, max_features=0.3, min_samples_leaf=10, n_estimators=300
Lc	gbm	l2_regularization=1.0, learning_rate=0.02, max_iter=200, max_leaf_nodes=63, min_samples_leaf=20
Lc	xgboost	colsample_bytree=0.6, learning_rate=0.02, max_depth=4, n_estimators=200, reg_lambda=5.0, subsample=1.0
Lc	lightgbm	colsample_bytree=0.8, learning_rate=0.02, n_estimators=200, num_leaves=31, reg_lambda=1.0, subsample=0.8
Lc	p2_hier_shrinkage	max_depth=8, n_estimators=500
Lr	random_forest	max_depth=14, max_features=0.3, min_samples_leaf=25, n_estimators=200
Lr	gbm	l2_regularization=0.0, learning_rate=0.02, max_iter=200, max_leaf_nodes=63, min_samples_leaf=20
Lr	xgboost	colsample_bytree=0.6, learning_rate=0.02, max_depth=4, n_estimators=300, reg_lambda=5.0, subsample=1.0
Lr	lightgbm	colsample_bytree=0.8, learning_rate=0.02, n_estimators=400, num_leaves=15, reg_lambda=1.0, subsample=0.6
Lr	p2_hier_shrinkage	max_depth=8, n_estimators=500

5.3 Calibration strategy

Every classifier is calibrated after fitting. The fitted estimator is frozen and an isotonic regression is fitted on the validation block, then applied unchanged to the test block and to the frozen-reference rows. Fitting one calibrator and reusing it for every matrix that must be scored by the same model is deliberate; refitting per matrix would silently produce two different calibrated models under one name. A sigmoid fallback and an uncalibrated fallback are available if isotonic fitting fails, and the method actually used is recorded per row. Isotonic succeeded everywhere in this run.

This strategy has a cost that Section 7 documents in full and that must be flagged here because it affects every headline number: isotonic calibration fitted on the validation block degrades both RPS and ECE for most models on this data. The reported tables use the calibrated probabilities throughout, because that is the model that would actually be served, but the uncalibrated score is retained in every results row so the cost is auditable.

5.4 Statistical testing protocol

Point estimates are not claims. Every pairwise comparison in this report is tested by a paired bootstrap clustered on match identifier. The per-match loss series of two models are paired, matches are resampled with replacement two thousand times, and the distribution of the mean paired difference gives a percentile confidence interval and a two-sided p-value. Clustering on match is essential for the in-play heads, where nineteen correlated snapshots share one label; treating them as independent rows would shrink the interval by roughly a factor of four and manufacture significance that is not there.

Because the zoo generates many pairs, all p-values are corrected by the Holm-Bonferroni step-down procedure within each task, and a comparison is reported as significant only if it survives that correction at the five per cent level. A separate seed-repetition study refits each model under three random seeds and reports whether an observed gap exceeds the seed-to-seed noise; that study is a stability diagnostic and is never used to license a superiority claim on its own.

6. Results: Models 1, 2 and 3

Every number in this section comes from one modelling run completed on 30 August 2026, in which all fifty-one pipeline stages exited zero. All classification metrics are computed on calibrated probabilities on the held-out test block, which the model selection process never touched.

6.1 Model 1 - Pre-match outcome classification (Task C)

Table 12. *Task C probabilistic results on the 5,808-match test block, ranked by RPS. The final column is the uncalibrated score, retained so that the cost of calibration is auditable.*

Model	RPS	Log-loss	Brier	ECE (calibrated)	RPS (uncalibrated)
xgboost	**0.19451**	**0.97159**	**0.57713**	**0.01960**	**0.19421**
p2_hier_shrinkage	0.19461	0.97071	0.57711	0.02175	0.19488
stack_temporal	0.19463	0.97191	0.57744	0.02875	0.19399
random_forest	0.19481	0.97195	0.57755	0.01303	0.19477
gbm	0.19491	0.97346	0.57817	0.01795	0.19471
stack	0.19524	0.97382	0.57897	0.02692	0.19432
lightgbm	0.19573	0.97714	0.58040	0.02745	0.19537
kernel_svm	0.20054	0.99381	0.59179	0.02805	0.20239
dummy	0.23008	1.07544	0.65087	0.01131	0.23039

Two things stand out and neither is what a reader would expect. The first is how tightly the field is packed. Excluding the dummy and the kernel machine, seven models span 0.19451 to 0.19573, a range of 0.00122 RPS, or roughly six tenths of one per cent of the score. Against a dummy at 0.23008 the models are unambiguously skilful - every one of them clears the floor by more than 0.034 RPS - but they are not meaningfully distinguishable from each other. The natural reading is that the pre-match signal available in this feature set is largely shared: once a learner has the rating layer, it has almost everything, and the choice of how to exploit it is nearly irrelevant. Section 6.6 confirms this statistically, and Section 8E confirms it a second way through the ablation.

The second is the kernel machine. kernel_svm is the only model outside the cluster, at 0.20054, and it is worse than every tree by a margin that does survive correction. It is also, by an enormous distance, the most expensive model in the zoo. Section 8D shows it needed 1,230 seconds to fit against xgboost's 2.2 seconds and 2,004 microseconds per row to serve against xgboost's 0.86. Paying five hundred times the fit cost for the worst score is the clearest single argument in this report that model-family choice on tabular data of this size should default to boosted trees.

Table 13. *Task C per-class precision, recall and F1 on the test block. Class supports are 2,509 home wins, 1,487 draws and 1,812 away wins. The draw column is the story.*

Model	P(H)	R(H)	F1(H)	P(D)	R(D)	F1(D)	P(A)	R(A)	F1(A)
xgboost	0.535	0.838	0.653	0.500	0.001	0.003	0.537	0.555	0.546
p2_hier_shrinkage	0.537	0.823	0.650	0.259	0.005	0.009	0.527	0.562	0.544
stack_temporal	0.541	0.818	0.651	0.286	0.003	0.005	0.523	0.578	0.549
random_forest	0.543	0.816	0.652	0.000	0.000	0.000	0.520	0.584	0.550

Model	P(H)	R(H)	F1(H)	P(D)	R(D)	F1(D)	P(A)	R(A)	F1(A)
gbm	0.537	0.829	0.652	0.000	0.000	0.000	0.531	0.567	0.548
stack	0.537	0.819	0.649	0.250	0.001	0.001	0.521	0.568	0.544
lightgbm	0.542	0.806	0.648	0.308	0.003	0.005	0.510	0.581	0.543
kernel_svm	0.540	0.815	0.649	0.000	0.000	0.000	0.511	0.570	0.538
dummy	0.432	1.000	0.603	0.000	0.000	0.000	0.000	0.000	0.000

Table 13 exposes what RPS conceals. No pre-match model predicts draws. Draw recall is between 0.000 and 0.005 for every learner, so on 1,487 drawn fixtures the models collectively identify a handful. This is not a defect of any one algorithm; it is the correct decision-theoretic response to the data. Draws are 25.6 per cent of the test block but are never the modal class for any input configuration, because a fixture that is finely balanced still has more probability mass on 'one side wins' than on 'exactly level'. An argmax rule will therefore never select D, and any model that did would be worse under every proper scoring rule. The right conclusion is that argmax accuracy is the wrong lens for this problem and that RPS, which rewards putting the correct amount of mass on D without demanding that it be the largest, is the metric the brief was right to prioritise. Section 8C tests the alternative - forcing draw predictions by class balancing - and measures exactly what it costs.

6.2 Model 2 - Pre-match goal-margin regression (Task R)

Table 14. *Task R results on the test block, ranked by mean absolute error. The correlation column separates ranking skill from scale.*

Model	MAE (goals)	RMSE	Correlation
gbm	**1.25035**	**1.60544**	**0.48675**
xgboost	1.25035	1.60612	0.48596
lightgbm	1.25087	1.60555	0.48606
stack_temporal	1.25123	1.60419	0.48666
stack	1.25139	1.60505	0.48613
random_forest	1.25270	1.60837	0.48362
p2_hier_shrinkage	1.25284	1.60737	0.48418
kernel_ridge_nystroem	1.26106	1.62194	0.47234
kernel_svr	1.26287	1.62876	0.47060
kernel_ridge_exact	1.26941	1.63601	0.46147
dummy	1.43711	1.83806	0.00000

The pattern repeats with one addition. The seven tree-based models span 1.25035 to 1.25284 in mean absolute error, a range of 0.0025 goals, against a constant-prediction dummy at 1.43711; the models remove roughly thirteen per cent of the dummy's error. The three kernel methods now form a clearly separated worse tier at 1.26106 to 1.26941, and here the separation **is** statistically real: Section 6.6 records fifteen significant Task R pairs and every one of them is a tree beating a kernel, with no tree-versus-tree pair separating at all.

The ordering within the kernel tier is itself informative. kernel_ridge_nystroem, an approximation, beats kernel_ridge_exact, the thing it approximates, by 0.0084 MAE, and the difference survives

Holm correction at p = 0.019. That looks paradoxical until the training sizes are compared: the exact solver is capped at 8,000 rows because the full Gram matrix would require roughly twelve gigabytes, while the Nystroem version consumes all 39,707. Five times the data beats an exact solution of a smaller problem. This is a concrete instance of the scaling argument developed in Section 8D and it is the most useful thing the kernel tier contributes to the report.

A correlation near 0.487 with a mean absolute error near 1.25 goals is the expected signature of a well-behaved football margin model. The models rank fixtures competently but predict conservatively: predictions concentrate near the home-advantage mean and rarely venture beyond plus or minus two goals, because emitting an extreme margin is punished heavily when the match is close and rewarded only rarely. Section 8A shows that all ten of the worst Task R errors are exactly this failure mode.

6.3 Model 3 - In-play prediction (Tasks Lc and Lr)

Table 15. *Task Lc probabilistic results aggregated over all nineteen snapshot minutes and 344 test matches (6,536 rows).*

Model	RPS	Log-loss	Brier	ECE (calibrated)	RPS (uncalibrated)
random_forest	**0.14559**	**0.78434**	**0.45109**	**0.04952**	**0.14318**
stack	0.14688	0.79347	0.45452	0.04652	0.14492
xgboost	0.14793	0.79322	0.46114	0.04855	0.14599
p2_hier_shrinkage	0.14841	0.81263	0.46327	0.03870	0.15212
stack_temporal	0.14945	0.80796	0.46409	0.06798	0.14842
gbm	0.15545	0.84270	0.48846	0.05782	0.16679
lightgbm	0.15566	0.84272	0.48487	0.04206	0.15300
dummy	0.22812	1.05455	0.63585	0.05225	0.22817

Table 16. *Task Lc per-class precision, recall and F1 across all snapshots. Compare the draw column with Table 13: live evidence, unlike pre-match evidence, does identify draws.*

Model	P(H)	R(H)	F1(H)	P(D)	R(D)	F1(D)	P(A)	R(A)	F1(A)
random_forest	0.767	0.757	0.762	0.417	0.486	0.449	0.682	0.605	0.641
stack	0.756	0.766	0.761	0.410	0.464	0.436	0.692	0.603	0.645
xgboost	0.735	0.799	0.766	0.389	0.328	0.356	0.642	0.630	0.636
p2_hier_shrinkage	0.737	0.781	0.759	0.407	0.410	0.409	0.691	0.616	0.651
stack_temporal	0.761	0.761	0.761	0.399	0.473	0.433	0.684	0.582	0.629
gbm	0.688	0.813	0.746	0.337	0.251	0.287	0.640	0.575	0.606
lightgbm	0.701	0.834	0.762	0.411	0.271	0.327	0.632	0.600	0.616
dummy	0.485	1.000	0.654	0.000	0.000	0.000	0.000	0.000	0.000

Table 17. *Task Lr results aggregated over all snapshot minutes, ranked by mean absolute error.*

Model	MAE (goals)	RMSE	Correlation
stack	**0.95633**	**1.27417**	**0.73971**
random_forest	0.96015	1.27988	0.73615
stack_temporal	0.96655	1.28535	0.73708
p2_hier_shrinkage	0.97757	1.30798	0.72114
xgboost	0.98435	1.29317	0.73632
gbm	0.98820	1.31036	0.72158
lightgbm	0.99445	1.30109	0.73061
dummy	1.48997	1.89230	0.00000

The in-play heads are a different regime entirely, and the comparison with the pre-match heads is the most instructive result in the report. Task Lc reaches RPS 0.14559 against Task C's 0.19451 and Task Lr reaches MAE 0.95633 against Task R's 1.25035, both on far less training data - 17,024 rows from 896 matches, against 39,707 matches. Watching the match is worth more than forty times the pre-match sample.

Table 16 makes the mechanism visible. Draw recall rises from at most 0.005 pre-match to 0.486 in play, with a draw F1 of 0.449 for the random forest. The reason is structural rather than algorithmic: at minute 80 with the score level, a draw genuinely **is** the modal outcome, so argmax selects it, and the class that was undetectable before kick-off becomes routinely detectable once elapsed time constrains it. This is also why the aggregate in-play numbers must be read with care - they average a nearly hopeless prediction at minute 0 with a nearly trivial one at minute 90. Section 8B decomposes them minute by minute, which is the only honest way to present them.

6.4 Ensembling

Table 18. *Best stacking variant against the best single learner on each head. Delta is the stack's score minus the best single learner's, so a positive value means the stack is worse. None of these differences survives correction.*

Task	Metric	Stack score	Best stack	Best single	Single score	Delta	Stack wins
C	RPS	0.19463	stack_temporal	xgboost	0.19451	+0.00012	no
Lc	RPS	0.14688	stack	random_forest	0.14559	+0.00129	no
R	MAE	1.25123	stack_temporal	gbm	1.25035	+0.00088	no
Lr	MAE	0.95633	stack	random_forest	0.96015	-0.00382	yes

The stack loses on three heads and wins on one, but the sizes involved are the point: the largest advantage anywhere is 0.0038 goals of mean absolute error on Task Lr, and the largest deficit is 0.0013 RPS on Task Lc. Section 6.6 tests all four comparisons and returns a Holm-corrected p of 1.000 in every case. Ensembling does not beat the best single model on any head, and with 5,808 test matches for the pre-match heads that null result now carries real statistical power rather than being a failure to detect an effect in a small sample.

One genuine ensemble finding does survive. On Task C, stack_temporal beats stack by 0.00060 RPS at Holm-corrected p = 0.048. That is the design argument of Section 5.1 measured rather than argued: fitting the meta-learner on recent validation data transfers better to a forward test block than fitting it on out-of-fold predictions spanning 2008 to 2021. The honest qualifier must be given in the same breath - neither stack beats plain xgboost, so the finding is about how to build a stack, not about whether to.

6.5 Comparison against the de-vigged market

This is the comparison the project exists to make. The models and the market are scored on exactly the same 5,806 test fixtures - the two Pro League matches without a tagged price are excluded from both sides rather than imputed - with the identical metric and no cherry-picking.

Table 19. *Task C models against the de-vigged Bet365 baseline on 5,806 tagged test matches. A positive gap means the model is worse than the market.*

Model	RPS	Log-loss	Brier	ECE	Gap to market
MARKET (de-vigged Bet365)	**0.18945**	**0.95433**	**0.56590**	**0.01817**	**-**
xgboost	0.19444	0.97144	0.57703	0.01944	+0.00499
p2_hier_shrinkage	0.19454	0.97055	0.57700	0.02159	+0.00509
stack_temporal	0.19456	0.97175	0.57733	0.02877	+0.00511
random_forest	0.19473	0.97178	0.57743	0.01304	+0.00528
gbm	0.19483	0.97329	0.57806	0.01797	+0.00538
stack	0.19516	0.97366	0.57887	0.02677	+0.00571
lightgbm	0.19567	0.97699	0.58030	0.02729	+0.00622
kernel_svm	0.20045	0.99360	0.59165	0.02807	+0.01100
dummy	0.23005	1.07541	0.65084	0.01117	+0.04060

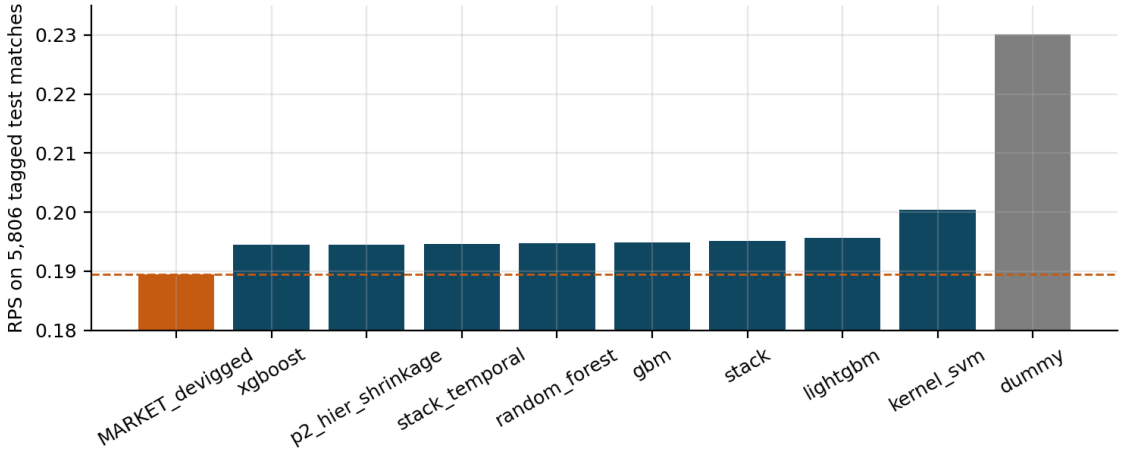

Figure 2. *Task C RPS on the 5,806 tagged test matches. The dashed line is the de-vigged market. Every model clears the dummy comfortably; every model sits above the market line, and the seven tree-based models are visually indistinguishable from one another.*

The market is not beaten, and the gap is 0.00499 RPS. The beats_market flag is false for all nine models. That is the headline result of the project and it is reported rather than buried, in line with the brief's standard that a well-analyzed failure to beat the market is worth more than an unexplained score that looks good.

The size of the gap is worth normalizing, because 0.005 in isolation means little. Taking the dummy as zero skill and the market as the attainable reference, the best model captures

$$\text{skill share} = \frac{0.23005 - 0.19444}{0.23005 - 0.18945} = \frac{0.03561}{0.04060} = 0.877$$

so roughly 88 per cent of the market's skill over a naive baseline is recovered by an odds-free model trained entirely on public data. That is a substantially stronger statement than the raw gap suggests, and it answers RQ1 directly. It also puts an upper bound on what better modelling could achieve here: the remaining twelve per cent is what the bookmaker knows that this feature set does not, which plausibly includes team news, injuries, suspensions, motivation and money flow - none of which is in any of the four data sources.

One further detail in Table 19 deserves comment because it validates the draw analysis of Section 6.1. The market's own draw recall is 0.00067, lower than several models. A market that prices draws correctly still almost never makes the draw its most likely outcome, which is exactly the decision-theoretic argument made earlier, now confirmed by an agent with money at stake.

6.6 Are the observed differences real?

Table 20. *Match-clustered paired bootstrap after Holm-Bonferroni correction. The first three columns exclude the trivial dummy comparisons; the last column is the total including them.*

Task	Pairs tested	Significant	Rate	Significant incl. dummy
C	28	10	35.7%	18
R	45	15	33.3%	25
Lc	21	1	4.8%	8
Lr	21	2	9.5%	9
All	**115**	**28**	**24.3%**	**60**

Table 21. *The pairwise comparisons that carry an argument, with match-clustered bootstrap confidence intervals and Holm-corrected p-values. The first four rows are the ensemble-versus-best-single tests.*

Task	Comparison	Mean difference	95% CI	Raw p	Holm p	Verdict
C	stack_temporal vs xgboost	+0.00013	[-0.00028, +0.00053]	0.5245	1.000	not separable
R	gbm vs stack_temporal	-0.00088	[-0.00288, +0.00121]	0.4185	1.000	not separable
Lc	random_forest vs stack	-0.00129	[-0.00383, +0.00131]	0.3450	1.000	not separable
Lr	random_forest vs stack	+0.00382	[-0.00365, +0.01163]	0.3280	1.000	not separable
C	lightgbm vs xgboost	+0.00123	[+0.00055, +0.00190]	0.0010	0.020	significant
C	lightgbm vs stack_temporal	+0.00110	[+0.00046, +0.00172]	0.0005	0.011	significant
C	stack vs stack_temporal	+0.00060	[+0.00019, +0.00101]	0.0025	0.048	significant
Lc	lightgbm vs random_forest	+0.01007	[+0.00361, +0.01641]	0.0020	0.042	significant
Lr	lightgbm vs stack	+0.03812	[+0.01737, +0.05795]	0.0000	0.000	significant
Lr	stack vs xgboost	-0.02802	[-0.04488, -0.01232]	0.0015	0.030	significant
R	kernel_ridge_exact vs kernel_ridge_nystroem	+0.00835	[+0.00405, +0.01312]	0.0005	0.018	significant

Three conclusions follow. First, all four ensemble-versus-best-single tests sit at Holm p = 1.000, so the point estimates in Table 18 are noise. The stack's significant Task Lr wins are over lightgbm and xgboost, which rank seventh and fifth on that head; beating the weak members of one's own ensemble is a statement about them, not a case for the ensemble. Second, the only significant non-trivial Task C results are that xgboost and stack_temporal beat lightgbm, and that stack_temporal beats stack; the top five Task C models are mutually inseparable. Third, on Task R all fifteen significant pairs are trees beating kernels or one kernel beating another; no tree-versus-tree difference separates.

The overall pattern - twenty-eight significant results out of 115 non-trivial pairs, concentrated entirely in comparisons across model families rather than within them - is the statistical form of the same sentence that Sections 6.1 and 6.2 reached descriptively: the choice between well-tuned tabular learners does not matter on this problem; the choice of information does.

6.7 Converting the Model 2 margin into Model 1 probabilities

If the margin and the outcome are two views of the same event, a margin forecast should be convertible into an outcome forecast. Two link functions were fitted on the validation block and applied to the test block. The ordinal link treats the realized result as a thresholded latent margin:

$$P(A) = \Phi\left(\frac{-\theta - m}{\sigma}\right), \quad P(H) = 1 - \Phi\left(\frac{\theta - m}{\sigma}\right), \quad P(D) = 1 - P(H) - P(A)$$

with theta and sigma estimated by maximum likelihood on validation, giving theta = 0.5412 and sigma = 1.5207. The alternative is a multinomial logistic regression on the predicted margin and its square. Both are compared against the directly trained classifier, with the base learner for each route selected on validation and all three scored on identical test rows.

Table 22. *Margin-to-probability conversion against the directly trained classifier, n = 5,808. All three routes are scored on uncalibrated probabilities so the comparison is like-for-like.*

Route	Source	RPS	Log-loss	Brier	ECE
model2_ordinal[lightgbm]	**Task R margin**	**0.19386**	**0.96826**	**0.57580**	**0.01481**
model2_logistic[lightgbm]	Task R margin	0.19396	0.96842	0.57591	0.01658
model1_direct[xgboost]	Task C classifier	0.19421	0.97040	0.57655	0.01227

Predicting the margin and converting it beats classifying directly, by 0.00035 RPS for the ordinal link. That is a genuine surprise and it has a plausible mechanism: the regression head is trained on an eleven-valued ordered target and therefore receives a much richer error signal per example than a three-class classifier does, so it learns the ordering of the outcome space more efficiently. The conversion then imposes the ordinal structure explicitly instead of asking the classifier to discover it. The effect closes about seven per cent of the remaining distance to the market. It must be labelled suggestive rather than established: this comparison was not among the pairs covered by the bootstrap study of Section 6.6, and a difference of 0.00035 is the same order as the differences that Section 6.6 found indistinguishable.

6.8 Odds as a feature: the declared blending arm

Odds are the evaluation baseline everywhere else in this report, so using them as an input would invalidate the comparison. One arm is nevertheless declared and reported separately, because the question 'how much would the price add if we were allowed it?' is worth an answer. The blend weight is selected on the validation block over a fixed grid and then applied unchanged to test. For the in-play head the weight decays exponentially with elapsed minute, since a pre-match price becomes less relevant as the match unfolds.

$$p_{blend} = \alpha \cdot p_{market} + (1 - \alpha) \cdot p_{model} \quad (pre\text{-}match); \quad \alpha(t) = \alpha_0 \, exp(-t/\tau) \quad (in\text{-}play)$$

Table 23. *The declared odds-as-feature arm. Blend weights are chosen on validation and applied to test; this arm is excluded from every other comparison in the report.*

Task	Arm	Base model	Alpha	Tau (min)	n	RPS
C	model_only	xgboost	-	-	5806	0.19444
C	market_only	xgboost	-	-	5806	0.18945
C	blend	xgboost	0.66	-	5806	0.19035
Lc	model_only	random_forest	-	-	6517	0.14572
Lc	blend	random_forest	0.34	30	6517	0.14235

The blend improves on the model alone in both heads: Task C falls from 0.19444 to 0.19035 at alpha = 0.66, and Task Lc from 0.14572 to 0.14235 at alpha_0 = 0.34 with a thirty-minute half-life. Two readings follow. First, the pre-match blend never reaches the market alone at 0.18945, so on Task C the model contributes nothing the price does not already contain - the optimal use of the model, given the price, is to weight it at only a third. Second, and more interesting, the in-play blend does beat both of its components: a pre-match price combined with live evidence is better than either, which is precisely what a live-betting desk would expect and is the one place in this report where the model demonstrably adds information the market did not have at kick-off.

7. Calibration and Reliability Analysis

7.1 The effect of calibration on the pre-match and in-play classifiers

Table 24. *Effect of isotonic calibration fitted on the validation block. Negative deltas are improvements. Calibration helps three of fourteen model-task pairs on RPS and four of fourteen on ECE.*

Task	Model	RPS raw	RPS calibrated	Delta RPS	ECE raw	ECE calibrated	Delta ECE
C	xgboost	0.19421	0.19451	+0.00030	0.01227	0.01960	+0.00733
C	p2_hier_shrinkage	0.19488	0.19461	-0.00027	0.01366	0.02175	+0.00809
C	stack_temporal	0.19399	0.19463	+0.00064	0.01142	0.02875	+0.01733
C	random_forest	0.19477	0.19481	+0.00004	0.01598	0.01303	-0.00295
C	gbm	0.19471	0.19491	+0.00020	0.01794	0.01795	+0.00001
C	stack	0.19432	0.19524	+0.00092	0.01347	0.02692	+0.01345
C	lightgbm	0.19537	0.19573	+0.00036	0.01661	0.02745	+0.01084
C	kernel_svm	0.20239	0.20054	-0.00185	0.04665	0.02805	-0.01860
Lc	random_forest	0.14318	0.14559	+0.00241	0.01383	0.04952	+0.03569
Lc	stack	0.14492	0.14688	+0.00196	0.02699	0.04652	+0.01953
Lc	xgboost	0.14599	0.14793	+0.00194	0.03412	0.04855	+0.01443
Lc	p2_hier_shrinkage	0.15212	0.14841	-0.00371	0.09795	0.03870	-0.05925
Lc	stack_temporal	0.14842	0.14945	+0.00103	0.05491	0.06798	+0.01307
Lc	gbm	0.16679	0.15545	-0.01134	0.16043	0.05782	-0.10261
Lc	lightgbm	0.15300	0.15566	+0.00266	0.05942	0.04206	-0.01736

The expectation before running this stage was routine: isotonic regression fitted on held-out data is a standard, near-free improvement to probability quality. What was observed is the opposite on this data, and it is worth stating without softening. Calibration degrades RPS for eleven of the fourteen model-task pairs and degrades ECE for ten of them. The damage is worst on the in-play head: the random forest's ECE rises from 0.01383 to 0.04952, a factor of three and a half, while its RPS rises from 0.14318 to 0.14559.

There are two mechanisms and they are separable. The first is distribution shift. The isotonic map is estimated on the 2021/22-2022/23 validation block and applied to 2023/24-2024/25; a monotone map fitted on one era is not the correct map for another, and isotonic regression, being non-parametric and step-shaped, is far more capable of encoding era-specific idiosyncrasy than a two-parameter sigmoid would be. The second, which applies to the in-play head, is correlation: the in-play validation block contains 277 matches presented as 5,263 highly correlated snapshot rows, so the effective sample behind the isotonic fit is roughly a twentieth of its nominal size and the fitted steps are badly over-determined.

A third mechanism affects one model specifically. stack_temporal fits its meta-learner on the earliest sixty per cent of the validation block, and the calibrator is then fitted on the whole of it. The isotonic map is therefore estimated partly on rows where the meta-learner's outputs are in-sample and consequently over-confident. That is consistent with what is measured: stack_temporal has the best uncalibrated Task C score in the entire zoo at 0.19399, ahead of xgboost's 0.19421, and calibration demotes it to third at 0.19463 while nearly trebling its ECE. The ranking in Table 12 is in this narrow sense an artefact of the calibration stage.

The calibrated numbers are nevertheless the ones reported throughout, because they describe the model that would actually be served and because switching to the uncalibrated column after seeing the scores would be selection on the test block. The correct remedy is architectural rather than presentational: the calibrator should be fitted on a slice of the validation block disjoint from any slice used for meta-learning, and a sigmoid link should be preferred to isotonic when the effective sample is small. Both are recorded in Section 9.4.

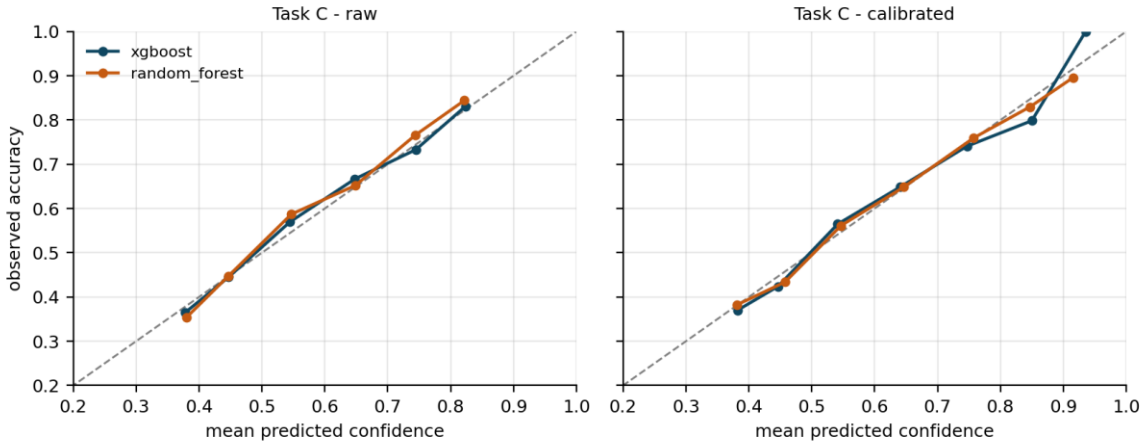

Figure 3. *Task C reliability diagrams before and after isotonic calibration. Points on the diagonal are perfectly calibrated. The raw curves already track the diagonal closely; calibration moves the low-confidence bins further from it.*

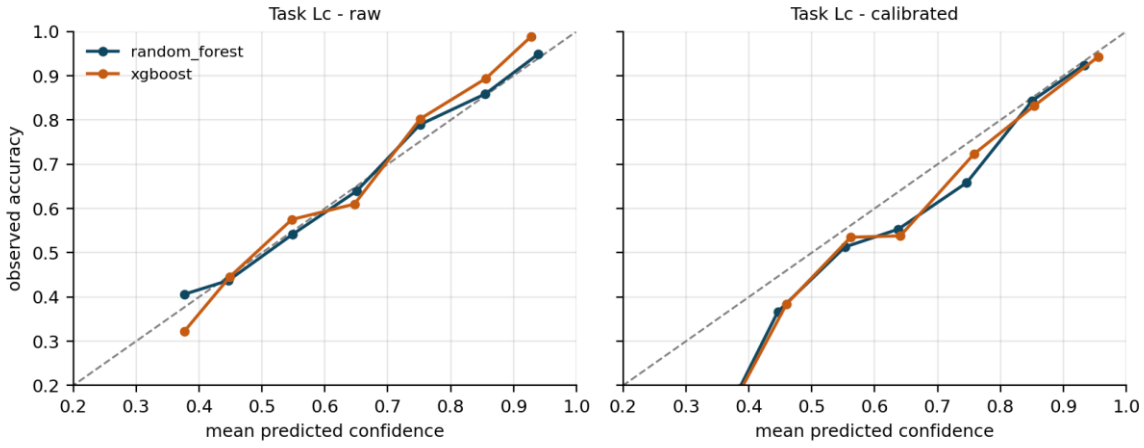

Figure 4. *Task Lc reliability diagrams before and after isotonic calibration. The high confidence bins, which are the late-match snapshots, are systematically under-confident after calibration - the model is right more often than it claims.*

Figures 3 and 4 show the same story geometrically. On Task C the raw curves sit close to the diagonal across the whole confidence range, which is why there is little for a calibrator to correct and why an over-flexible one does harm. On Task Lc the calibrated curve rises above the diagonal at high confidence: where the model says eighty-five per cent it is right rather more often than that. Under-confidence is the benign direction of miscalibration - it costs score but never produces a confident wrong claim - and it is the signature of a monotone map fitted on early-match rows being applied to late-match rows where the problem is much easier.

7.2 Per-phase calibration for Model 3

Table 25. *Model 3 calibration by game phase for the served random forest and the boosted contrast. Over-confidence is mean confidence minus accuracy, so a negative value means the model is under-confident.*

Model	Phase (min)	n	Mean confidence	Accuracy	Over-confidence	ECE	RPS
random_forest	0-15	1032	0.5696	0.4758	+0.0939	0.09516	0.21402
random_forest	15-30	1032	0.6133	0.5155	+0.0978	0.10412	0.20317
random_forest	30-45	1032	0.6436	0.5678	+0.0758	0.07576	0.18027
random_forest	45-60	1032	0.7093	0.6453	+0.0639	0.06500	0.14320
random_forest	60-75	1032	0.7795	0.7355	+0.0441	0.05437	0.11181
random_forest	75-90	1376	0.8446	0.8910	-0.0464	0.05748	0.05219
xgboost	0-15	1032	0.5636	0.4632	+0.1004	0.10106	0.22001
xgboost	15-30	1032	0.6057	0.5078	+0.0979	0.09790	0.20613
xgboost	30-45	1032	0.6430	0.5611	+0.0820	0.08199	0.18112
xgboost	45-60	1032	0.7084	0.6444	+0.0640	0.06403	0.14304
xgboost	60-75	1032	0.7680	0.7229	+0.0451	0.04514	0.11048
xgboost	75-90	1376	0.8157	0.8772	-0.0615	0.06924	0.05708

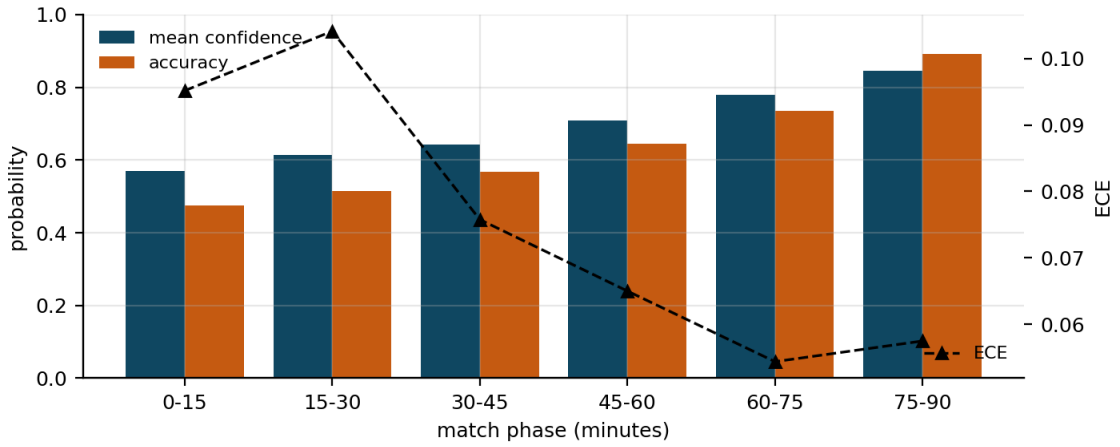

Figure 5. *Mean confidence against realised accuracy by match phase for the served in-play random forest, with ECE overlaid. The bars cross between the 60-75 and 75-90 phases: the model switches from over-confident to under-confident in the closing quarter.*

The aggregate ECE of 0.04952 in Table 15 is an average over two opposite errors, and Table 25 separates them. Through the first five phases the model is over-confident, by between 0.098 and 0.044, and the over-confidence shrinks monotonically as the match progresses. In the closing quarter the sign flips: mean confidence is 0.845 against an accuracy of 0.891, an under-confidence of 0.046.

Both halves of that pattern have the same cause, which is the calibrator again. A single monotone map is fitted across all nineteen snapshot minutes at once, so it must compromise between the early-match regime, where confident predictions are frequently wrong, and the late-match regime, where confident predictions are nearly always right. The compromise necessarily over-corrects one end and under-corrects the other. The obvious remedy, and the one recommended in Section 9.4, is a per-phase or minute-conditioned calibrator, which the existing per-phase table already provides the diagnostic for.

The RPS column also documents the difficulty gradient that Section 8B quantifies in detail: the same model scores 0.21402 in the opening quarter hour and 0.05219 in the last, a fourfold change driven entirely by how much of the match remains uncertain.

8A. Error and Failure Analysis

8A.1 Global attributions

Table 26. *Mean absolute SHAP value per feature, the six largest for the served model on each head. Attributions are computed with TreeSHAP over the test block.*

Task	Model	Rank	Feature	Mean |SHAP|
C	xgboost	1	pi_expected_gd	0.09507
		2	elo_diff	0.05978
		3	elo_expected_home	0.04951
		4	diff_squad_overall_top18	0.03104
		5	diff_squad_overall_top11	0.02566
		6	elo_home_pre	0.01651
R	lightgbm	1	pi_expected_gd	0.20467
		2	elo_diff	0.15192
		3	elo_expected_home	0.11602
		4	diff_squad_overall_top11	0.05914
		5	diff_form_stat_shots_a	0.04154
		6	diff_squad_overall_top18	0.03914
Lc	random_forest	1	inplay_rate_diff_goals	0.05788
		2	inplay_diff_goals	0.05773
		3	inplay_goal_diff	0.05095
		4	inplay_away_goals	0.01518
		5	pi_expected_gd	0.01494
		6	inplay_away_xg	0.00852
Lr	random_forest	1	inplay_diff_goals	0.24510
		2	inplay_rate_diff_goals	0.23253
		3	inplay_goal_diff	0.20383
		4	inplay_home_goals	0.06867
		5	inplay_away_goals	0.05920
		6	pi_expected_gd	0.05068

The pre-match heads are dominated completely by the rating layer. On Task C the top three features - pi_expected_gd, elo_diff and elo_expected_home - account for more attribution than the next fifty combined, and the fourth and fifth are the squad-quality differences. Not one rolling-form feature reaches the top five. This is the expected result and it is worth saying why: Elo and pi-ratings are themselves recursive summaries of the entire match history of both clubs, so they compress into two numbers what the form window approximates with forty. Given the ratings, the form columns are largely redundant, and the ablation in Section 8E confirms this from the other direction by showing that removing them costs almost nothing.

The in-play heads invert the picture entirely. The top three features on both Lc and Lr are the live goal difference in its three encodings - raw, differenced and rate-per-minute - and on Task Lr they carry roughly four times the attribution of the strongest pre-match feature. pi_expected_gd survives into fifth and sixth place, so the pre-match prior is not discarded, but it has been demoted from the whole story to a tie-breaker. Notably the in-play expected-goals proxy attracts far less attribution than the score itself, which foreshadows the ablation result that removing it **improves** the model.

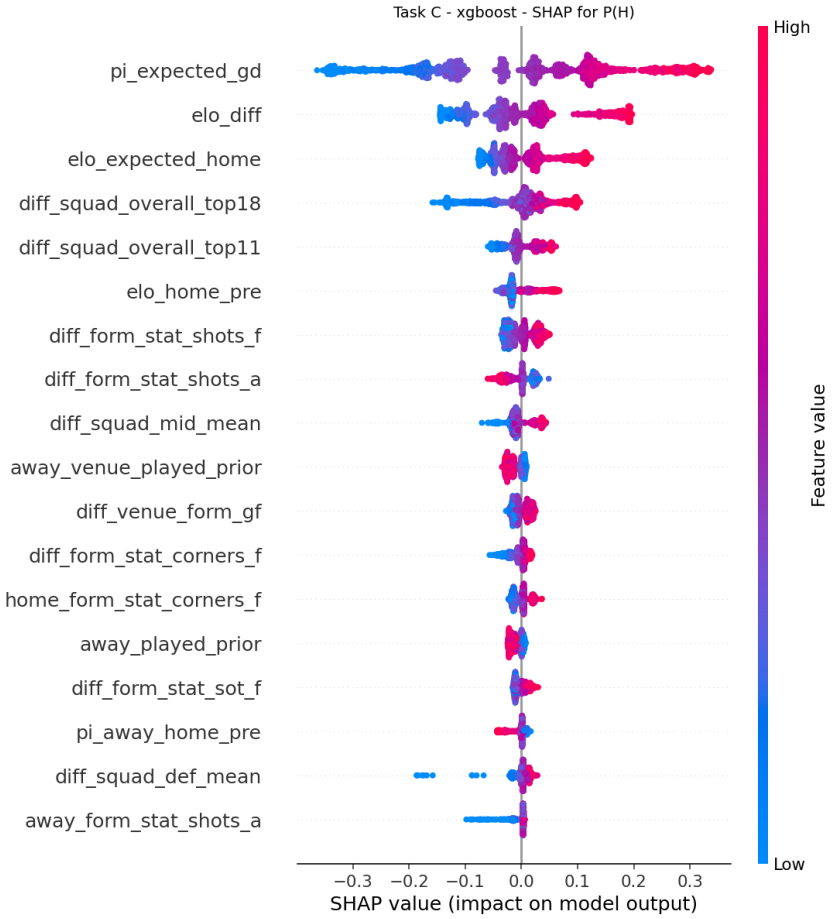

Figure 6. *Task C SHAP beeswarm for the served xgboost model, home-win class. Each point is one test match; horizontal position is the feature's contribution to the home-win log-odds and colour is the feature value. The near-linear red-to-blue sweep on pi_expected_gd shows the model using the rating monotonically, as it should.*

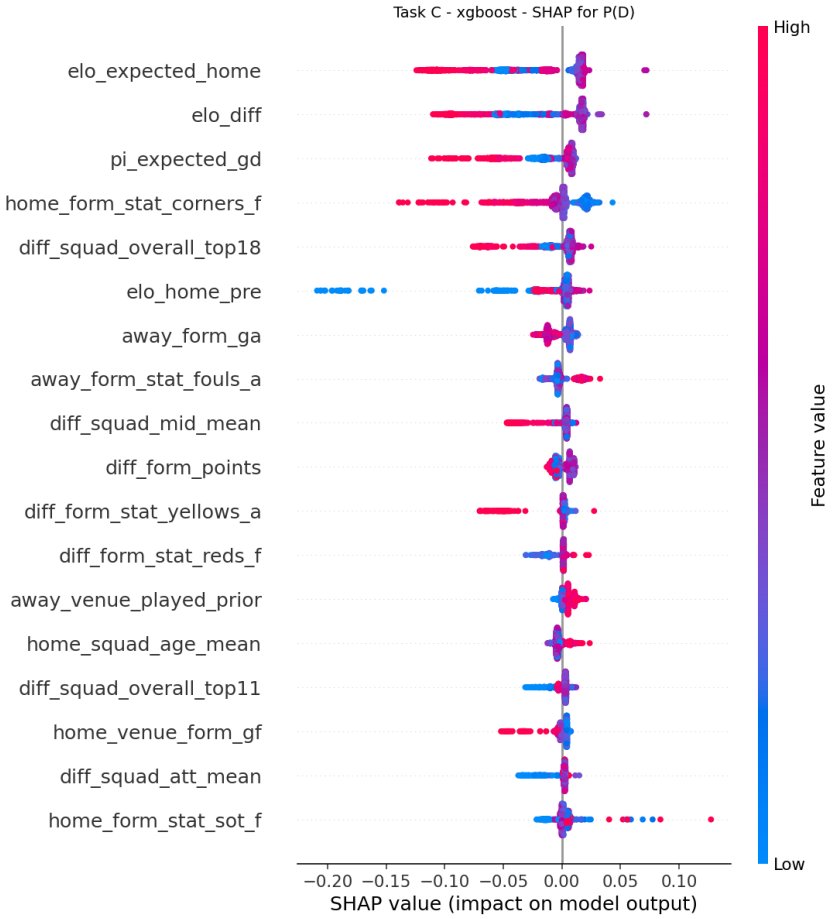

Figure 7. *Task C SHAP beeswarm, draw class. The contributions are an order of magnitude smaller than for the home class and are concentrated near zero, which is the attribution-level explanation of why no pre-match model ever selects the draw.*

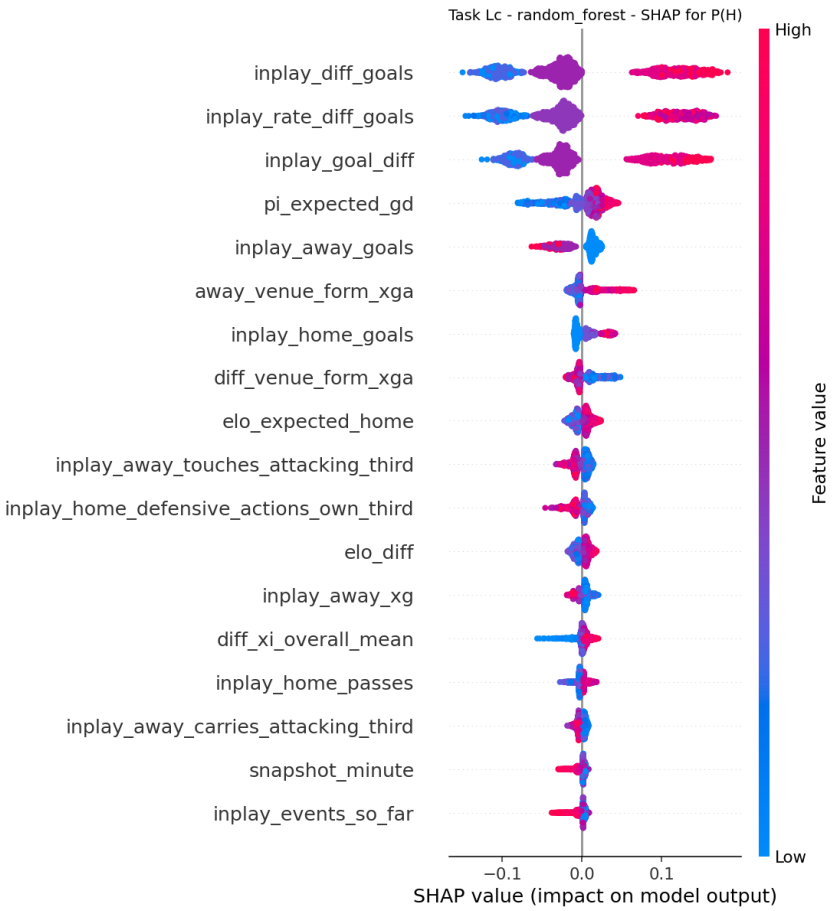

Figure 8. *Task Lc SHAP beeswarm for the served in-play random forest, home-win class. The live goal-difference features separate cleanly into discrete bands, one per scoreline state, which is the visual signature of a model that has learned the scoreboard rather than a continuous trend.*

8A.2 The ten worst predictions per task

Table 27. *The five worst predictions per task, adjudicated. The market column shows the de-vigged price assigned to the realized outcome by the bookmaker on the same fixture, where one exists.*

Task	Rank	Match id	Minute	Loss	Actual	P(actual)	Market P(actual)	Verdict
C	1	2000024703	-	0.7864	A	0.0622	0.0632	inherently uncertain
C	2	2000002300	-	0.7832	A	0.0737	0.0451	inherently uncertain
C	3	2000019672	-	0.7806	A	0.0711	0.0551	inherently uncertain
C	4	2000016303	-	0.7784	A	0.0730	0.0797	inherently uncertain
C	5	2000024574	-	0.7729	A	0.0672	0.0938	inherently uncertain
R	1	2000005375	-	6.4540	5.0	-	-	inherently uncertain
R	2	2000005240	-	5.5185	-5.0	-	-	inherently uncertain
R	3	2000005413	-	5.4472	-5.0	-	-	inherently uncertain
R	4	2000005244	-	5.4391	-5.0	-	-	inherently uncertain
R	5	2000016479	-	5.4115	-4.0	-	-	inherently uncertain

Task	Rank	Match id	Minute	Loss	Actual	P(actual)	Market P(actual)	Verdict
Lc	1	3754064	45	0.9258	H	0.0097	0.3722	noisy observation
Lc	2	3754064	40	0.9176	H	0.0133	0.3722	noisy observation
Lc	3	3754064	35	0.9105	H	0.0138	0.3722	noisy observation
Lc	4	3754064	30	0.9025	H	0.0138	0.3722	noisy observation
Lc	5	3754064	55	0.9003	H	0.0176	0.3722	noisy observation
Lr	1	3879847	0	4.8273	-4.0	-	-	inherently uncertain
Lr	2	3879847	5	4.8124	-4.0	-	-	inherently uncertain
Lr	3	3879847	10	4.7768	-4.0	-	-	inherently uncertain
Lr	4	3754333	5	4.5316	4.0	-	-	inherently uncertain
Lr	5	3754333	0	4.4859	4.0	-	-	inherently uncertain

The adjudication is the point of this table, and it separates two very different kinds of failure. Thirty of the forty worst predictions are labelled inherently uncertain: the model assigned low probability to the realized outcome, and so did the market. On the ten worst Task C errors the model's probability for the realized away win ranged from 0.062 to 0.080, while the bookmaker's ranged from 0.045 to 0.094 - on several of them the market was more wrong than the model. These are not modelling failures. They are matches in which a heavy home favorite lost, and a forecaster that had assigned them appreciable probability would have been worse on the other several thousand fixtures. Reporting them as errors without this comparison would misrepresent them.

All ten worst Task R errors share one signature: the realized margin was at or near the clipping bound of plus or minus five, and the prediction sat between minus 1.5 and plus 1.4. This is the conservatism identified in Section 6.2, and it is the correct behavior under mean absolute error. A model that predicted more extreme margins would reduce these ten losses and increase several thousand others.

The remaining ten, all on the in-play heads, are labelled noisy observation and are more interesting because they concentrate in a single fixture. Eight of the ten worst Task Lc errors are different snapshots of match 3754064, at minutes 25 through 60, where the model assigned the eventual home win a probability between 0.010 and 0.018 while the pre-match market had it at 0.372. That is the fingerprint of a match in which the home side was losing heavily at the half and then recovered. Because the loss is computed per snapshot, a single such comeback contributes nineteen correlated rows to the error tail and dominates it. This is exactly why the significance testing in Section 6.6 clusters on match: an uncorrected analysis would treat this one fixture as eight independent pieces of evidence.

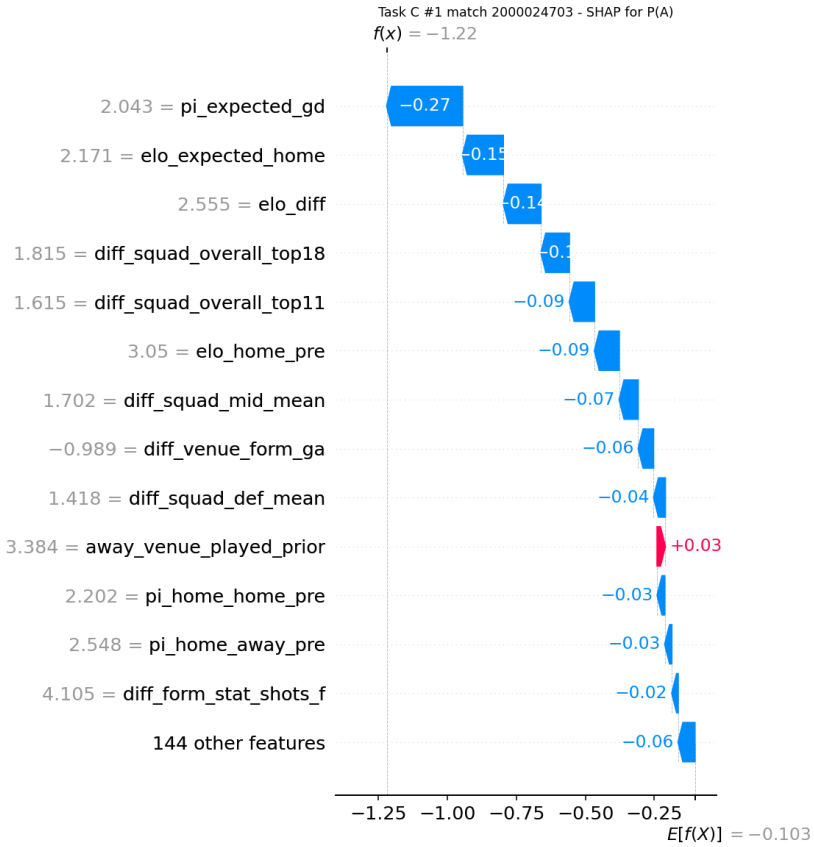

Figure 9. *The single worst Task C prediction. The model's probability for the realised away win was 0.062; the de-vigged market's was 0.063. Both forecasters were equally surprised, which is the definition of an irreducible error.*

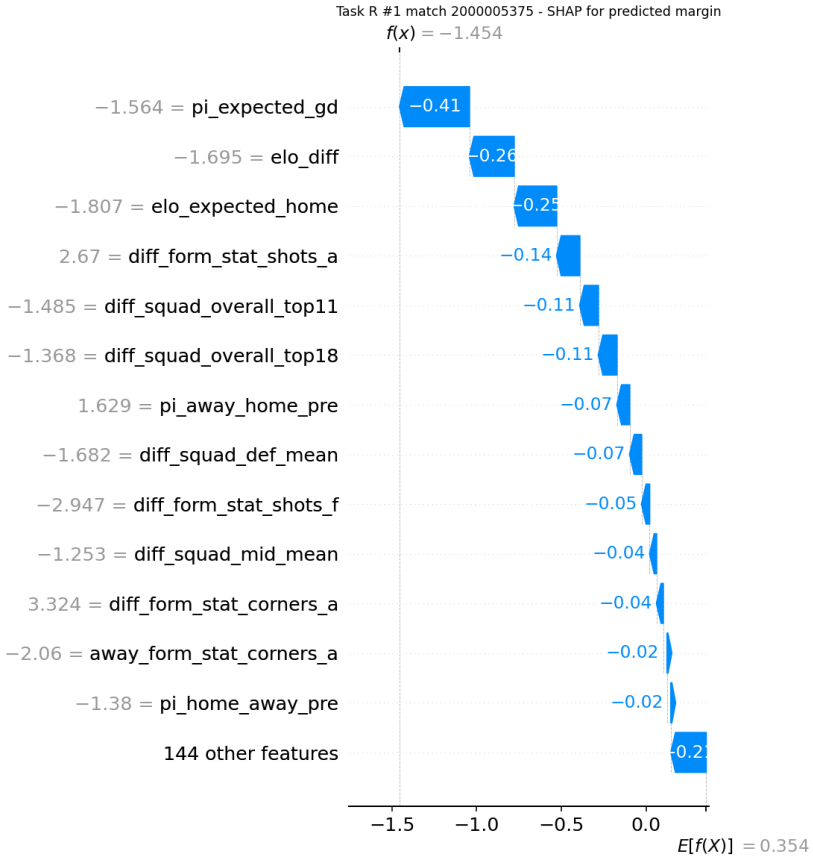

Figure 10. *The single worst Task R prediction: a realised margin at the +5 clipping bound against a prediction of -1.45. Every one of the ten worst regression errors has this shape.*

8B. In-Play Analysis

The aggregate in-play scores of Section 6.3 average nineteen very different prediction problems. This section decomposes them, and answers RQ2.

Table 28. *In-play performance against the frozen pre-match reference at every snapshot minute, for both in-play heads on the same 344 test matches. A negative delta means live evidence has overtaken the pre-match forecast.*

Minute	Lc frozen RPS	Lc in-play RPS	Delta	Lr frozen MAE	Lr in-play MAE	Delta
0	0.20444	0.21427	+0.00983	1.33441	1.34529	+0.01088
5	0.20444	0.21186	+0.00742	1.33441	1.31134	-0.02307
10	0.20444	0.21593	+0.01149	1.33441	1.30269	-0.03172
15	0.20444	0.20762	+0.00318	1.33441	1.25444	-0.07997
20	0.20444	0.20388	-0.00056	1.33441	1.24719	-0.08722
25	0.20444	0.19801	-0.00643	1.33441	1.21722	-0.11719
30	0.20444	0.19222	-0.01222	1.33441	1.19512	-0.13929
35	0.20444	0.18155	-0.02289	1.33441	1.12376	-0.21065
40	0.20444	0.16705	-0.03739	1.33441	1.05861	-0.27580
45	0.20444	0.15304	-0.05140	1.33441	0.99378	-0.34063
50	0.20444	0.14413	-0.06031	1.33441	0.93302	-0.40139
55	0.20444	0.13243	-0.07201	1.33441	0.88643	-0.44798
60	0.20444	0.12091	-0.08353	1.33441	0.82359	-0.51082
65	0.20444	0.11078	-0.09366	1.33441	0.77290	-0.56151
70	0.20444	0.10373	-0.10071	1.33441	0.72413	-0.61028
75	0.20444	0.08437	-0.12007	1.33441	0.65498	-0.67943
80	0.20444	0.06204	-0.14240	1.33441	0.56591	-0.76850
85	0.20444	0.03957	-0.16487	1.33441	0.44689	-0.88752
90	0.20444	0.02278	-0.18166	1.33441	0.38555	-0.94886

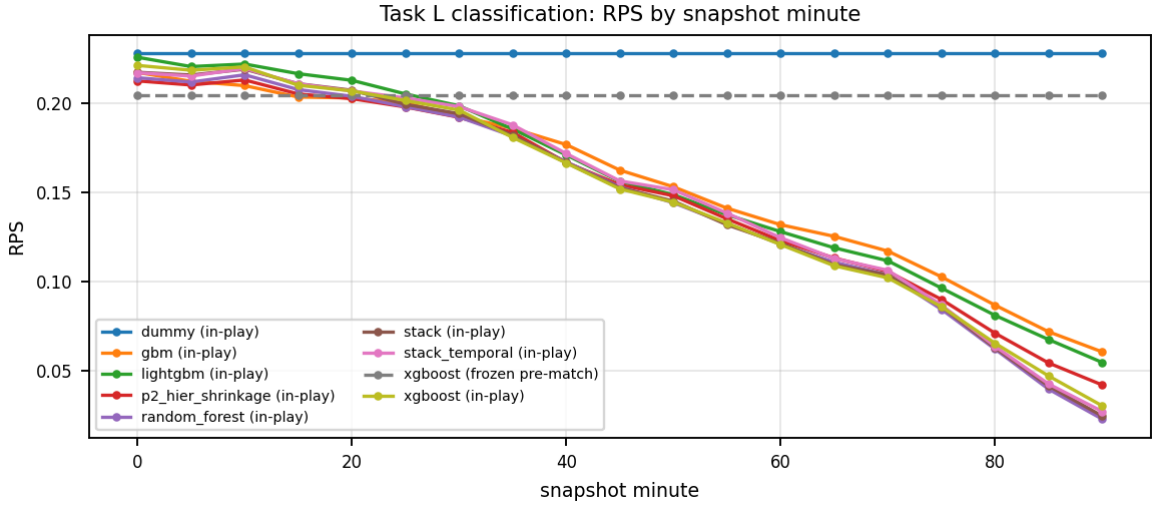

Figure 11. *Task Lc ranked probability score against snapshot minute, with the frozen pre-match reference and the dummy floor. The in-play curve starts above the reference and crosses it at minute 20.*

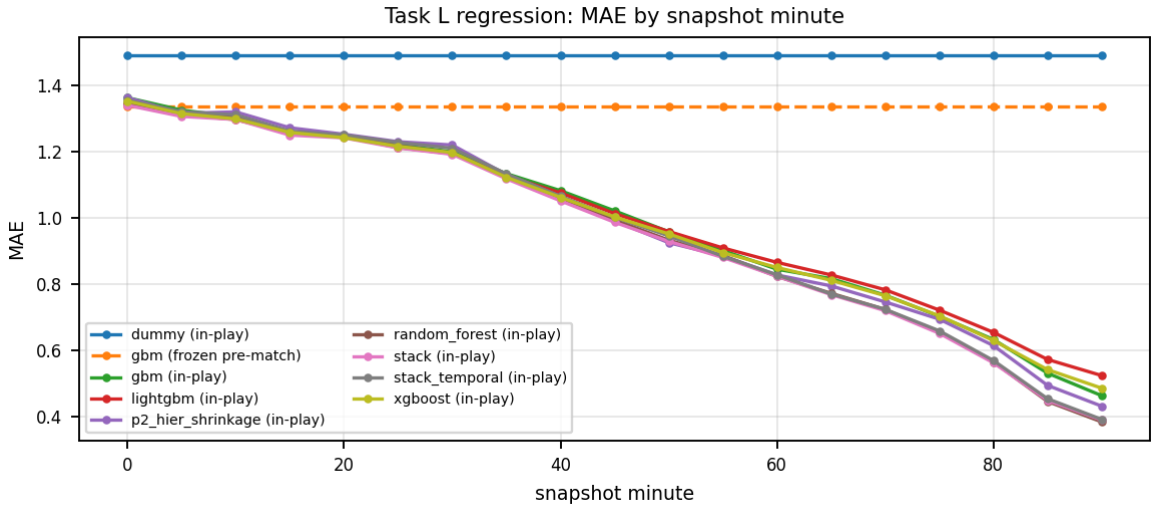

Figure 12. *Task Lr mean absolute error against snapshot minute. The crossover is much earlier here, at minute 5, because a single goal constrains the margin far more sharply than it constrains the outcome.*

The in-play model starts worse than the pre-match model and does not overtake it until minute 20. At kick-off Task Lc scores 0.21427 against the frozen reference's 0.20444. This is initially counter-intuitive - the minute-0 snapshot contains the pre-match vector, so how can it be worse? - and the explanation is entirely about training data. The in-play model is fitted on 17,024 rows from 896 matches of a single 2015/16 season, while the frozen reference is fitted on 39,707 matches spanning thirteen seasons. At minute 0 the in-play model has no live evidence to compensate for that forty-four-fold disadvantage in sample. Parity arrives at minute 20, half-time reads 0.15304 against 0.20444, and the final snapshot reaches 0.02278, a ninefold improvement on the reference.

The margin head crosses far earlier, at minute 5. The asymmetry is real and informative: knowing that one goal has been scored moves the expected margin immediately and by a large amount, whereas it moves the outcome distribution much less, because a one-goal lead at minute 5 leaves eighty-five minutes for the result to change. Information about the magnitude of the result arrives earlier than information about its direction.

One caveat must accompany the crossover minute, and it makes the claim conservative rather than optimistic. The frozen reference is the served pre-match model scored on the 621 excluded StatsBomb fixtures. Those fixtures are genuinely held out of pre-match training - that exclusion is what the excluded split exists for - but they date from 2015/16, whereas the model that scores them was fitted on data running to 2021. The reference is therefore anachronistic in the model's favor: it benefits from seasons that postdate the matches it is grading. A strictly forward pre-match reference would be weaker, so the true crossover would arrive earlier than minute 20, not later.

8B.1 Which in-play features carry the gain

Table 29. *Task Lc feature-group ablation, validation block, uncalibrated. Every row is a full refit with the named group removed.*

Configuration	Columns	Validation RPS	Delta
full	**394**	**0.12596**	**0.00000**
prematch_only	240	0.19279	+0.06683
drop_inplay_score	388	0.17380	+0.04784
drop_pi_ratings	393	0.12700	+0.00104

Configuration	Columns	Validation RPS	Delta
drop_inplay_tempo	386	0.12678	+0.00082
drop_inplay_shots	377	0.12664	+0.00068
drop_elo	392	0.12684	+0.00088
drop_inplay_passing	374	0.12653	+0.00057
drop_inplay_xg	387	0.12539	-0.00057

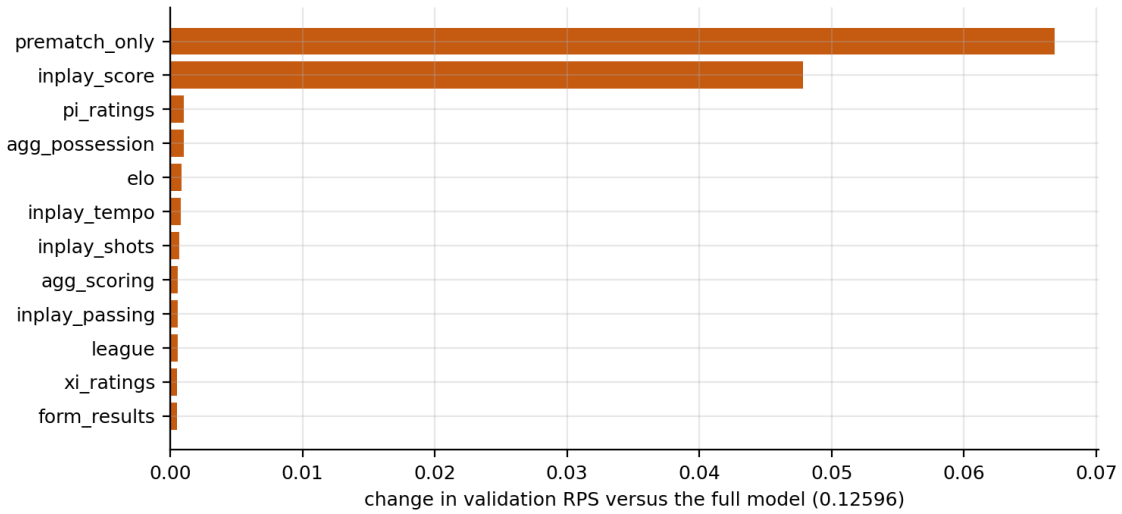

Figure 13. *Task Lc feature-group ablation, twelve most consequential groups. Bars to the right are groups whose removal hurts; bars to the left are groups the model is better without.*

This is the cleanest single result in the run. Removing the live score costs 0.048 RPS; removing every pre-match rating costs less than 0.002; removing in-play expected goals gains 0.0006. Stripping the in-play block entirely returns 0.19279, essentially the pre-match score, which confirms the two pipelines are wired correctly. At five-minute granularity the scoreline is very nearly a sufficient statistic for the in-play problem, and the elaborate territory, pressure and possession blocks - roughly 130 columns between them - are close to free parameters. Three of those groups have negative deltas, meaning the model is measurably better without them.

The expected-goals result deserves emphasis because it contradicts the intuition that motivated building the feature. Shot-quality information should in principle tell you that a team leading 1-0 has been fortunate. Measured, it does not: layered on top of an exact scoreline, the in-play xG proxy adds noise. The plausible reason is that the proxy is computed from shot location alone and is coarse enough that its signal is dominated by the variance it introduces.

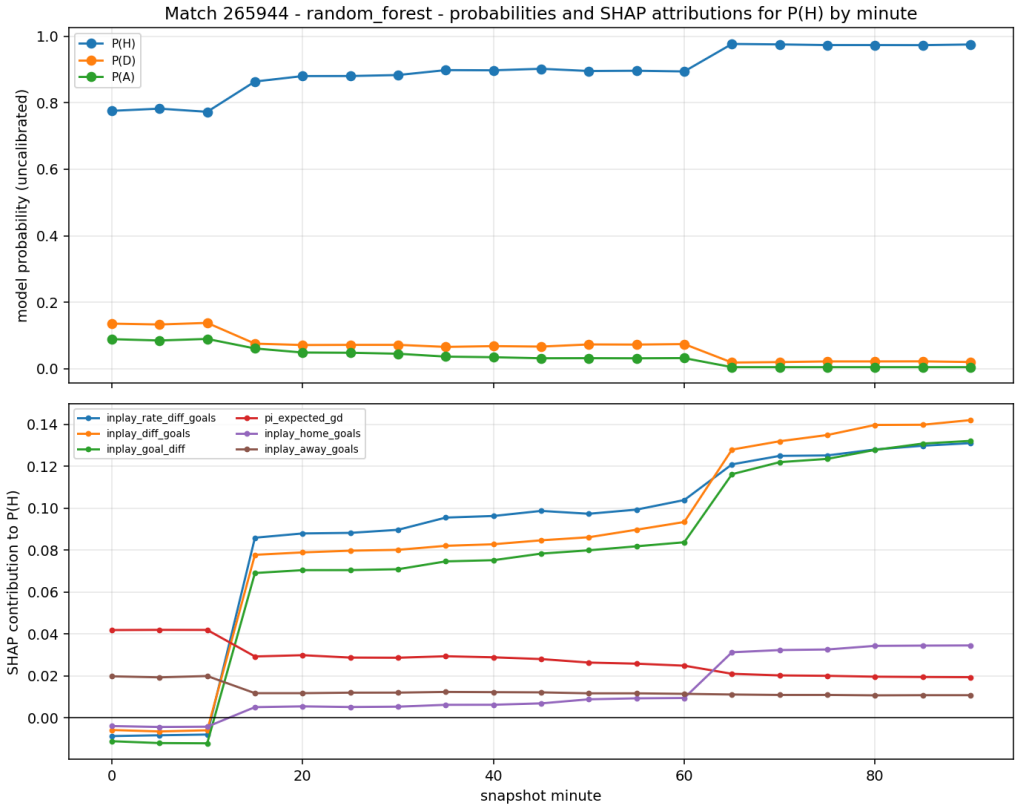

Figure 14. *SHAP attribution across the nineteen snapshots of a single complete match. The goal-difference band expands sharply at each scoring event and the pre-match rating contribution shrinks monotonically as the match progresses.*

Figure 14 shows the same transition within one fixture. Early snapshots are dominated by pi_expected_gd and elo_diff, the pre-match prior. As goals are scored the live goal-difference features take over, and by the closing snapshots the rating contribution has shrunk to a rounding error. The model is not switching between two behaviours; it is continuously reweighting one feature set against another as evidence accumulates, which is exactly what the in-play design was intended to produce.

8C. Imbalance and Resampling

The training block contains 39,707 matches of which the draw class is the minority, and Section 6.1 established that no pre-match model predicts draws at all. Six arms were compared to test whether that can be fixed and what fixing it costs. Four learners are run through each arm; the vanilla rows are bit-identical to the corresponding rows of Table 12, which confirms the study shares its pipeline with the main sweep.

Table 30. *Six-arm imbalance comparison on Task C, test block, calibrated. Best and mean RPS are over the four study learners; the recall and F1 columns give the best draw detection any learner in the arm achieved.*

Arm	Train rows	Best learner	Best RPS	Mean RPS	Best draw recall	Best draw F1
class_weight	39707	gbm	0.19475	0.19631	0.0195	0.0364
p1_gsmotenc	54108	gbm	0.19477	0.19629	0.0242	0.0440
vanilla	39707	random_forest	0.19481	0.19650	0.0027	0.0053
smote	54108	lightgbm	0.19519	0.19725	0.0081	0.0157
borderline_smote	54108	lightgbm	0.19550	0.19748	0.0215	0.0402
adasyn	56390	random_forest	0.19575	0.19836	0.0188	0.0357

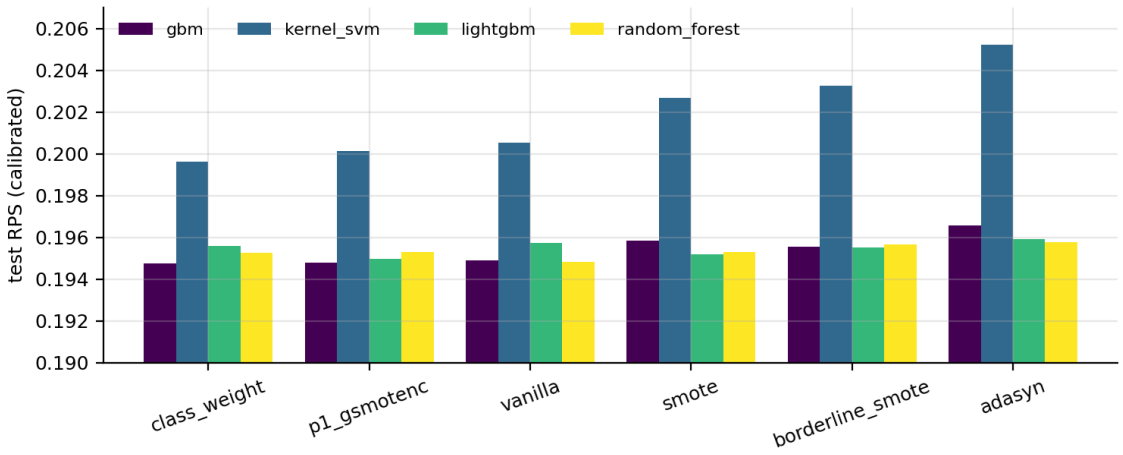

Figure 15. *Task C test RPS by resampling arm and learner. Every synthetic-oversampling arm sits above the vanilla arm for at least three of the four learners.*

The ordering is unambiguous: plain class reweighting and G-SMOTENC are indistinguishable from doing nothing, and all three synthetic oversampling methods are worse. The best arm, class_weight at 0.19475, beats vanilla at 0.19481 by 0.00006 RPS, a margin that Section 6.6's methodology would place far inside the noise. SMOTE, borderline-SMOTE and ADASYN cost between 0.0004 and 0.0009 RPS, which are the largest effects in the table and are all in the wrong direction.

One row of Table 30 is not a real arm and should be discounted: xgboost under class_weight returns exactly its vanilla score, because the library does not accept that parameter and silently ignores it. The class-weight arm is therefore effectively a three-learner arm.

8C.1 What resampling actually does to the draw class

The post-calibration draw recall in Table 30 never exceeds 0.024, which would suggest resampling fails even at its own stated objective. That conclusion would be wrong, and separating the two stages shows why. Table 31 reports the same arms scored on the validation block **before** the calibrator is applied.

Table 31. *Balancing arms scored on the validation block without calibration. Compare the draw-recall column with Table 30: the gain is real, and the calibrator is what removes it.*

Arm	Learner	Train rows	Validation RPS	Draw recall	Draw F1
vanilla	random_forest	39707	0.19771	0.0000	0.0000
vanilla	xgboost	39707	0.19730	0.0000	0.0000
class_weight	random_forest	39707	0.20376	0.3594	0.3173
class_weight	xgboost	39707	0.19730	0.0000	0.0000
smote	random_forest	54108	0.20274	0.3112	0.2989
smote	xgboost	54108	0.19984	0.0405	0.0711
borderline_smote	random_forest	54108	0.20351	0.2456	0.2642
borderline_smote	xgboost	54108	0.20082	0.0272	0.0496
adasyn	random_forest	56390	0.20570	0.1619	0.2067
adasyn	xgboost	56390	0.20206	0.0146	0.0279

Class balancing does recover the draw class. A class-weighted random forest lifts draw recall from 0.000 to 0.359 with a draw F1 of 0.317, and SMOTE lifts it to 0.311. What then happens is that the isotonic calibrator, fitted on an unresampled validation block, maps the inflated draw probabilities back down to their empirical frequency - which is the correct behavior for a calibrator - and the argmax rule stops selecting D again. The two tables are not in conflict; they measure different points in the pipeline.

The honest summary is therefore three-part. Balancing buys draw recall; it costs RPS, between 0.005 and 0.008 on validation for the random forest; and calibration, which is applied for independent and good reasons, returns the recall to near zero while leaving most of the RPS cost in place. Unless draw recall is itself the deliverable, the correct decision is not to resample**.**

8C.2 The effect of P1, isolated

Table 32. *Paper P1 against no resampling, validation block, uncalibrated. G-SMOTENC costs RPS on both learners and recovers no draws on either.*

Learner	RPS without P1	RPS with P1	Delta	Draw recall without	Draw recall with	P1 helps
random_forest	0.19771	0.19839	+0.00068	0.0000	0.0000	no
xgboost	0.19730	0.19792	+0.00062	0.0000	0.0000	no

P1 costs 0.00068 RPS on the random forest and 0.00062 on xgboost, and lifts draw recall on neither. The comparison in Table 30 is slightly kinder - G-SMOTENC is the second-best arm there on the calibrated test block - but the two readings agree on the substantive point: the paper's method is not harmful, and it is not useful here either.

This should be reported as a fair test rather than a failed reimplementation, and the reason it fails is instructive. G-SMOTENC synthesizes minority points on a hyperball surface around existing minority examples, which helps when the minority region is under-sampled and the classifier cannot see its boundary. With 39,707 training matches and roughly ten thousand draws, that boundary is not under-sampled; it is genuinely diffuse. Draws are not a compact region of feature space that more examples would reveal - they are what happens when two similar teams meet, and the surrounding region is occupied by matches that ended one-nil in either direction. Fabricating more points inside a region that is intrinsically mixed adds noise rather than definition. The method addresses sample scarcity, and the problem here is not scarcity but overlap.

8D. Compute and Scaling Analysis

The brief asks for an empirical demonstration that kernel methods scale as their theory predicts. Two studies were run: a controlled scaling sweep on Task R, and a full profile of every model on every head.

Table 33. Raw scaling measurements against the dense-matrix memory bound. The Gram column is the storage an exact n-by-n kernel matrix would require at double precision.

Method	Train rows	Fit seconds	Peak memory (MB)	Gram matrix (MB)	Empirical exponent	Theoretical exponent
kernel_ridge_exact	500	0.0066	0.6	1.9	2.266	3.0
	1000	0.0120	1.3	7.6		
	2000	0.0450	2.0	30.5		
	4000	0.3383	245.8	122.1		
	8000	1.7290	977.1	488.3		
	12000	5.4482	747.5	1098.6		
	17024	13.1939	1841.9	2211.1		
kernel_svr_exact	500	0.0206	8.1	1.9	2.029	2.0
	1000	0.0698	6.9	7.6		
	2000	0.2587	20.1	30.5		
	4000	0.9782	62.9	122.1		
	8000	3.8324	206.2	488.3		
	12000	8.4082	239.2	1098.6		
	17024	36.7585	90.0	2211.1		
kernel_ridge_nystroem	500	0.0443	9.5	1.9	0.377	1.0
	1000	0.0329	8.8	7.6		
	2000	0.0409	15.4	30.5		
	4000	0.0562	30.6	122.1		
	8000	0.0820	2.1	488.3		
	12000	0.1052	0.0	1098.6		
	17024	0.1474	129.9	2211.1		

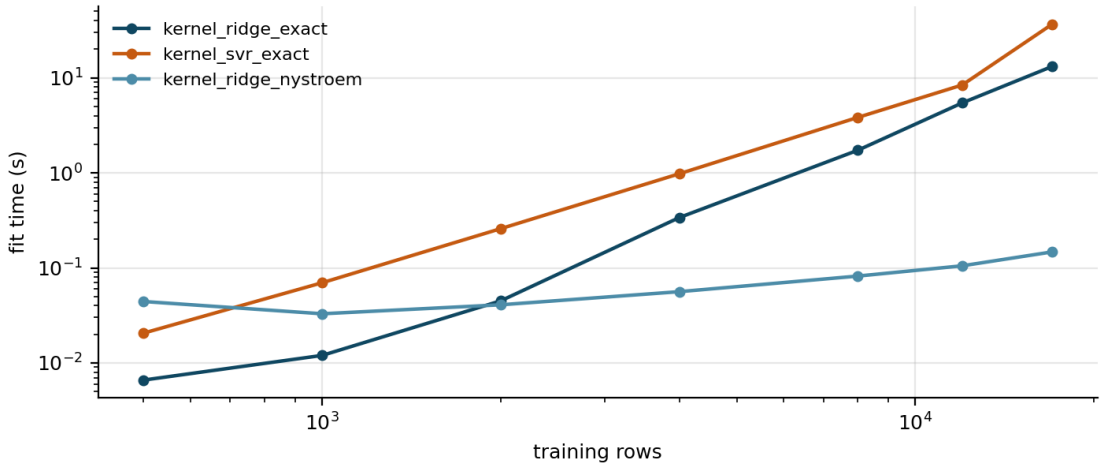

Figure 16. Fit time against training rows on log-log axes. A straight line is a power law and its slope is the scaling exponent. The two exact methods climb steeply; the Nystroem approximation is nearly flat.

Table 34. Empirical against theoretical scaling exponents, fitted by least squares on the log-log fit-time curve.

Method	Theoretical complexity	Theoretical exponent	Empirical exponent	Interpretation
kernel_ridge_exact	O(n^2) space, O(n^3) solve	3.000	2.266	Below theory: LAPACK exploits structure, and the 8,000-row cap truncates the regime where the cubic term dominates
kernel_svr_exact	O(n^2) space, ~O(n^2..3) SMO	2.000	2.029	Matches theory almost exactly
kernel_ridge_nystroem	O(nm) space, O(nm^2) fit	1.000	0.377	Below theory: with m fixed at 1,000 the cost is dominated by the fixed decomposition, so growth in n is nearly free

The memory column is the more decisive measurement. At 17,024 rows the exact Gram matrix already requires 2.2 GB; at the full pre-match training size of 39,707 it would require roughly twelve gigabytes, which is why kernel_ridge_exact is capped at 8,000 rows and reports a subsampled flag in its results row. That cap is not an implementation shortcut but the quadratic memory bound made concrete, and it is what causes the approximation to beat the exact method in Table 14.

8D.1 Full compute and memory profile

Table 35. Compute and memory profile for every model on every head. Serving cost is microseconds per test row, measured over the full test block.

Task	Model	Train rows	Features	Fit (s)	Peak fit (MB)	Model size (MB)	Predict (us/row)
C	dummy	39707	157	0.009	0.3	0.00	0.01
C	random_forest	39707	157	9.005	30.5	15.38	13.30
C	gbm	39707	157	4.367	3.2	0.81	5.85
C	kernel_svm	39707	157	1229.677	37.8	42.26	2004.41
C	p2_hier_shrinkage	39707	157	35.630	128.2	22.60	41.97
C	xgboost	39707	157	2.206	22.4	1.01	0.86
C	lightgbm	39707	157	4.988	34.2	2.06	8.07
R	dummy	39707	157	0.000	0.0	0.00	0.00
R	random_forest	39707	157	12.937	1.6	9.29	6.56
R	gbm	39707	157	1.962	0.8	0.50	3.08
R	kernel_svr	39707	157	44.895	46.9	45.03	2140.37
R	kernel_ridge_exact	8000	157	1.431	856.4	9.64	31.05
R	kernel_ridge_nystroem	39707	157	0.541	0.0	8.84	8.46
R	p2_hier_shrinkage	39707	157	82.168	1.7	6.88	13.85
R	xgboost	39707	157	0.449	14.2	0.23	0.19
R	lightgbm	39707	157	0.887	59.9	0.31	1.22
Lc	dummy	17024	394	0.004	0.0	0.00	0.01
Lc	random_forest	17024	394	15.588	1.3	5.90	4.13
Lc	gbm	17024	394	15.307	73.5	4.61	24.72
Lc	p2_hier_shrinkage	17024	394	20.478	30.6	14.19	40.48
Lc	xgboost	17024	394	2.806	19.9	0.93	0.68
Lc	lightgbm	17024	394	7.201	53.2	2.05	4.98
Lr	dummy	17024	394	0.000	0.0	0.00	0.00
Lr	random_forest	17024	394	12.452	1.8	7.54	4.62
Lr	gbm	17024	394	4.927	38.2	1.89	7.74
Lr	p2_hier_shrinkage	17024	394	259.793	0.6	14.06	36.45
Lr	xgboost	17024	394	1.354	82.3	0.49	0.38
Lr	lightgbm	17024	394	2.423	91.3	0.61	2.26

Three observations carry practical weight. First, the cost spread within a single head is enormous and is not correlated with quality: on Task C, xgboost fits in 2.2 seconds and serves in 0.86 microseconds per row for the best score in the zoo, while kernel_svm fits in 1,230 seconds and serves in 2,004 microseconds per row for the worst. That is a factor of 557 in fit time and 2,330 in serving latency, paid for a worse model. Second, the two stacking variants cost 300 and 58 seconds on Task C, because a stack must fit all of its members, and Section 6.6 showed they buy nothing. Third, p2_hier_shrinkage is between five and twenty times slower to fit than the boosted models, because lambda selection refits an internal cross-validation loop; the shrinkage itself is free, but the selection is not.

Taken together these numbers make the served-model choice straightforward and defend it on grounds independent of accuracy. The pre-match classifier is xgboost at 1.0 MB and 0.86 microseconds per row; the in-play classifier is the random forest at 5.9 MB and 4.13 microseconds per row.

Table 36. *Serving latency of the prediction service, in milliseconds, measured over repeated calls against the persisted model bundles.*

Endpoint	Calls	Mean	p50	p95	p99	Max
/health	300	0.276	0.267	0.333	0.409	0.525
/predict	300	0.338	0.328	0.377	0.478	1.497
/predict?pre	300	0.335	0.328	0.398	0.451	0.619
/replay	60	1.352	1.288	1.573	2.385	2.388

The service answers a pre-match or in-play prediction request with a p95 of 0.377 ms and a p99 of 0.478 ms, and a full nineteen-snapshot replay of one match at a p95 of 1.573 ms. Both are two to three orders of magnitude inside any plausible live-broadcast budget, which confirms that the model-selection decisions above were never latency-constrained.

8E. Theory versus Reality

8E.1 Ablation: which pre-match feature groups actually carry signal

Table 37. *Task C feature-group ablation on the validation block, uncalibrated, random forest. Positive deltas mean the group was carrying signal; negative deltas mean the model is better without it.*

Configuration	Columns	Validation RPS	Delta
drop_pi_ratings	152	0.19817	+0.00046
drop_elo	153	0.19797	+0.00026
drop_squad_ratings	130	0.19793	+0.00022
drop_form_goals	151	0.19782	+0.00011
drop_league	143	0.19779	+0.00008
drop_head_to_head	146	0.19778	+0.00007
drop_schedule	152	0.19774	+0.00003
drop_form_results	151	0.19773	+0.00002
full	157	0.19771	+0.00000
drop_xi_ratings	129	0.19769	-0.00002
drop_venue_form	143	0.19763	-0.00008

Table 38. *Task R feature-group ablation on the validation block, uncalibrated, random forest. Positive deltas mean the group was carrying signal; negative deltas mean the model is better without it.*

Configuration	Columns	Validation MAE	Delta
drop_squad_ratings	130	1.26834	+0.00271
drop_pi_ratings	152	1.26746	+0.00183
drop_head_to_head	146	1.26586	+0.00023
drop_elo	153	1.26582	+0.00019
drop_xi_ratings	129	1.26571	+0.00008
drop_form_results	151	1.26570	+0.00007
full	157	1.26563	+0.00000
drop_league	143	1.26557	-0.00006
drop_form_goals	151	1.26539	-0.00024
drop_venue_form	143	1.26517	-0.00046
drop_schedule	152	1.26506	-0.00057

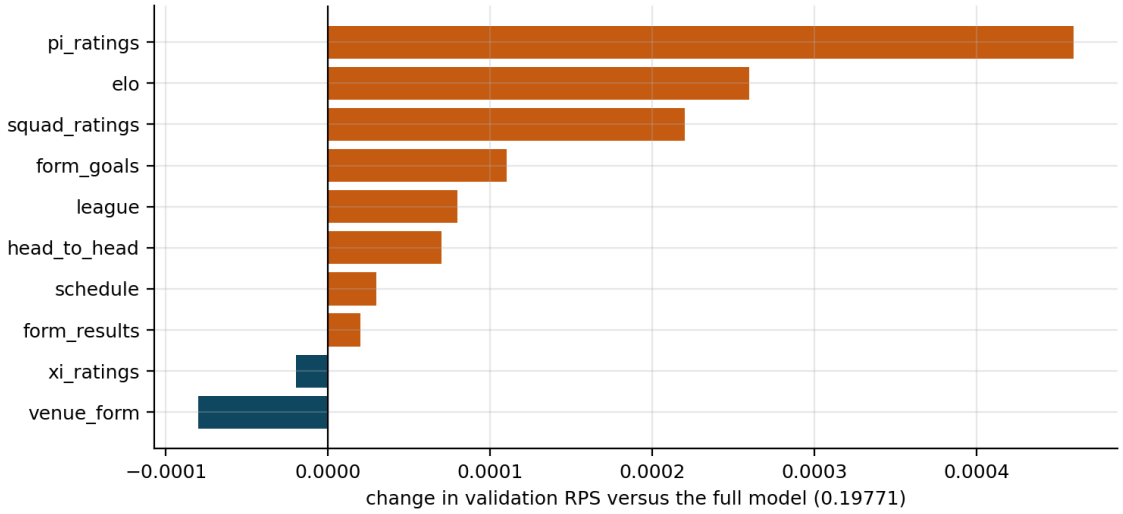

Figure 17. *Task C feature-group ablation, twelve largest absolute effects. Every effect is smaller than 0.0005 RPS, and two groups have negative deltas.*

A note on scale is needed before these tables are read against Section 6. The ablation is scored on the validation block with uncalibrated probabilities, because a feature study must not touch the test block and because refitting a calibrator for every one of nearly two hundred configurations would confound the feature effect with a calibration effect. The full-model Task C figure of 0.19771 is therefore not comparable with the 0.19451 of Table 12; the two differ by split and by calibration, not by model. Only the deltas within each table are meaningful.

The pre-match result is that no feature group matters very much and the rating layer matters most. Removing pi-ratings costs 0.00046 RPS, Elo 0.00026 and squad ratings 0.00022; removing all of the rolling-form families costs between 0.00002 and 0.00011. Two groups have negative deltas: venue form at -0.00008 and the XI ratings at -0.00002, the latter being the dead-column problem diagnosed in Section 3.4. On Task R the ordering is the same but the magnitudes are larger and squad ratings lead, costing 0.0027 goals of mean absolute error, which is consistent with squad quality bearing more directly on how many goals a team scores than on whether it wins.

The contrast with the in-play ablation of Section 8B is the substantive finding. There, one group - the live score - was worth 0.048 RPS, a hundred times the largest pre-match effect. Pre-match performance is information-limited, not capacity-limited. No rearrangement of these features will close the remaining gap to the market, because the features themselves are close to redundant with one another; only new information would.

8E.2 Snapshot frequency: when more data is not more information

Table 39. Snapshot frequency ablation. The training density is thinned while the validation set is held at full nineteen-snapshot density, so the comparison is like-for-like at evaluation time.

Task	Model	Configuration	Train snapshots/match	Train rows	Validation metric
Lc	random_forest	every_5_min	19	17024	0.12596
Lc	xgboost	every_5_min	19	17024	0.12976
Lc	random_forest	every_10_min	10	8960	0.12585
Lc	xgboost	every_10_min	10	8960	0.12875
Lc	random_forest	every_15_min	7	6272	0.12578
Lc	xgboost	every_15_min	7	6272	0.12812
Lr	random_forest	every_5_min	19	17024	0.83478
Lr	xgboost	every_5_min	19	17024	0.86208
Lr	random_forest	every_10_min	10	8960	0.81381
Lr	xgboost	every_10_min	10	8960	0.85492
Lr	random_forest	every_15_min	7	6272	0.81537
Lr	xgboost	every_15_min	7	6272	0.85000

The expectation was that denser training snapshots would help, since nineteen snapshots per match is nearly three times the density of seven. The measurement contradicts it: on Task Lc the random forest scores 0.12596 at five-minute density, 0.12585 at ten and 0.12578 at fifteen, so coarser is marginally better, and on Task Lr the fifteen-minute arm improves mean absolute error by 0.019 goals. The pattern holds for xgboost as well.

The explanation is that adjacent snapshots are almost duplicates. Between minute 40 and minute 45 most matches record no goal and no dismissal, so the two rows differ only in a handful of accumulated counters while sharing an identical label. Thinning the training set from nineteen to seven snapshots removes 65 per cent of the rows and almost none of the information, while reducing the effective duplication that biases the fit toward matches with many uneventful passages. The nineteen-snapshot design does not purchase resolution and the report does not claim that it does; it is retained because the deliverable specifies a forecast at every five minutes, which is a serving requirement rather than a training one.

8E.3 Other divergences between theory and measurement

- **Ensembling.** Theory says a stack of decorrelated members should beat its best member. Measured, it does not, on any of the four heads, with Holm-corrected p = 1.000 throughout. The members are not decorrelated: Section 8E.1 showed they are all reading the same two or three rating features, so there is little residual error for a meta-learner to combine away.
- **Approximation versus exactness.** Theory says an exact kernel solve dominates a rank-1,000 approximation of it. Measured, the approximation wins by 0.0084 MAE

at Holm p = 0.019, because the exact method could only be given a fifth of the training data within its memory bound.

- **Calibration.** Theory says post-hoc isotonic calibration on held-out data improves probability quality at negligible cost. Measured, it degrades RPS in eleven of fourteen cases and ECE in ten, for the distribution-shift and correlated-sample reasons set out in Section 7.1.
- **Synthetic oversampling.** Theory says minority oversampling improves minority detection. Measured, it does - draw recall rises from 0.000 to 0.359 - but the improvement does not survive calibration and the RPS cost does, so the net effect on the deliverable metric is negative.
- **Expected goals.** Theory says shot quality corrects the noise in a scoreline. Measured, on top of an exact live scoreline, the in-play xG proxy is noise itself: removing it improves Task Lc by 0.00057 RPS.

The common thread is that four of these five divergences are cases where a technique that helps in the small-sample or high-noise regime fails to help once the sample is large and the dominant feature is nearly sufficient. That is not a criticism of the techniques; it is a statement about which regime this problem occupies, and it is the most transferable lesson in the report.

9. Conclusion and Limitations

9.1 Objectives achieved

- Four public sources with no shared key were integrated into one relational store of 54,983 matches across eleven leagues and eighteen seasons, with a canonical registry of 358 clubs, a full alias map, a data-quality log and a verified download manifest of 93 files.
- Two feature pipelines were built from that store - a 157-column pre-match matrix and a 394-column in-play snapshot matrix - with four asserted leakage barriers.
- All three required models were delivered and evaluated on held-out data: pre-match outcome classification, pre-match margin regression, and in-play prediction at nineteen snapshots per match for both target types.
- Two papers were reimplemented from their published descriptions, tested, and reported honestly including where they did not help.
- Every model was compared against both a dummy floor and a de-vigged market ceiling on identical fixtures, with all pairwise claims tested by a match-clustered paired bootstrap under Holm-Bonferroni correction.
- Calibration, error analysis, SHAP attribution, ablation, resampling, scaling and serving latency were each measured and interpreted.

9.2 Key findings, answered against the research questions

RQ1 - how close to the market? The best odds-free model trails the de-vigged Bet365 price by 0.00499 RPS on 5,806 tagged test matches, and is beaten by it on every one of nine models. Normalised against the dummy, the model captures 87.7 per cent of the market's skill. The market remains unbeaten and that is reported as the project's headline result.

RQ2 - when does live evidence win? At minute 20 for the outcome head and minute 5 for the margin head. The in-play model starts **worse** than a frozen pre-match forecast, at 0.21427 against 0.20444, because it trains on forty-four times less data; it reaches 0.15304 by half-time and 0.02278 at the final snapshot. Because the frozen reference is anachronistic in its own favour, minute 20 is a conservative estimate.

RQ3 - does oversampling help the draw class? No. Every synthetic arm scores below doing nothing on the deliverable metric. Balancing genuinely does recover draw recall, from 0.000 to 0.359 uncalibrated, but the calibration stage removes the recall while leaving most of the RPS cost, and P1 specifically costs 0.0006 RPS and recovers no draws at all.

RQ4 - does hierarchical shrinkage help? Marginally and never decisively. P2 is second on two heads and fourth on two, and no tree-versus-tree difference on the pre-match heads is statistically separable. With 39,707 training rows and a tuned depth of eight, the thin-leaf regime the paper targets does not arise.

RQ5 - does ensembling beat the best single model? No, on none of the four heads, at Holm-corrected $p = 1.000$ in every case. One narrower result does survive: on Task C the temporally fitted meta-learner beats the out-of-fold one at $p = 0.048$, so the design argument for fitting on recent validation data is supported - but neither stack beats plain xgboost.

RQ6 - do kernel methods scale as predicted? Substantially yes. The exact support-vector regressor's empirical exponent of 2.029 matches its theoretical 2.0 almost exactly, and the quadratic memory bound is binding in practice: an exact Gram matrix at full training size

would need roughly twelve gigabytes, which forces a five-fold data cap that costs the exact method more than the approximation error costs its rival.

One finding sits outside the research questions and deserves its own sentence, because it is the most useful thing the report has to say about where effort should go next. Pre-match performance on this data is information-limited, not capacity-limited. Seven very different learners span 0.00122 RPS; no pre-match feature group is worth more than 0.0005 RPS when removed; and the largest single in-play feature effect is a hundred times larger than the largest pre-match one. More modelling will not close the market gap. More information might.

9.3 Limitations

- **The training block spans eleven leagues and the evaluation blocks span nine.** Ekstraklasa and the Swiss Super League contribute 3,342 training matches from competitions absent from validation and test, and carry no odds at all. The shift is measured - every league and era indicator scores below 0.0012 mean absolute SHAP - but it is not zero.
- **The XI-rating family is empty throughout validation and test.** Real lineups end with the European Soccer Database in 2016, so twenty-seven columns are median-imputed constants on the evaluation data. The ablation shows the model is very slightly better without them.
- **Squad ratings are frozen at FIFA 23 across the entire test window**, so the fourth and fifth most important Task C features are one to two years stale on 2024/25 fixtures.
- **Calibration degrades most reported scores.** Isotonic calibration fitted on the validation block worsens RPS in eleven of fourteen model-task pairs. The calibrated numbers are reported because they describe the served model, but the ranking in Table 12 is in part an artefact of this stage, and stack_temporal is the best Task C model before calibration and the third after it.
- **The stack_temporal calibrator overlaps its own meta-training rows.** The meta-learner is fitted on the earliest sixty per cent of validation and the calibrator on all of it, so the isotonic map is estimated partly on in-sample outputs.
- **Hyperparameters were searched on a subsample.** The grid search used at most 8,000 rows against a final training size of 39,707, so the selected configurations are not optimal for the size they are finally fitted at.
- **The in-play layer rests on a single season.** All event data comes from the four 2015/16 big-five seasons, giving 896 training and 344 test matches. Every in-play conclusion is therefore drawn from one season of one era, and the 344-match test block is small enough that only one non-trivial Task Lc pair separates statistically.
- **The frozen pre-match reference is anachronistic.** It scores 2015/16 fixtures with a model fitted on data running to 2021. The fixtures are properly held out of training, but the reference benefits from later seasons, so it is optimistic and the crossover minute is conservative.
- **The margin-to-probability result is untested.** It beats the direct classifier on a point estimate of 0.00035 RPS, which is the same order as differences the bootstrap study found indistinguishable, and it was not among the tested pairs.
- **The 2025/26 season is loaded but unused.** 2,903 matches sit in the store marked out of scope; rolling them into the evaluation window requires moving all four split constants together and re-running the model layer.

9.4 Future improvements

- **Fix the calibrator before anything else.** Split the validation block into disjoint slices for meta-learning and calibration, prefer a sigmoid link where the effective sample is small, and fit a minute-conditioned calibrator for the in-play head. Section 7.2 shows a single monotone map cannot serve both the opening and closing phases, and the per-phase table already provides the diagnostic.
- **Acquire current squad ratings.** Replacing the FIFA 23 carry-forward with genuine 2023/24 and 2024/25 rosters addresses the fourth and fifth most important pre-match features directly, and is the cheapest available route to new information.
- **Drop the XI family from the pre-match model** and retain it only in the in-play pipeline where lineups genuinely exist.
- **Extend the event-data layer beyond 2015/16.** Every in-play conclusion currently rests on 344 test matches; more seasons would both improve the in-play model - which Section 8B showed is data-starved at minute 0 - and give the significance tests real power on that head.
- **Add genuinely new pre-match information** rather than new pre-match models: team news, injuries, suspensions and rest-quality signals are the plausible content of the twelve per cent of market skill not currently recovered.
- **Retune at full training size.** The 8,000-row search cap most plausibly leaves the boosted models under-capacity, and retuning is a pure compute cost with no design risk.
- **Test the margin-to-probability route properly** by adding it to the bootstrap study, so that a promising point estimate becomes either a finding or a non-finding.

Appendix A. Hand Derivation of the P2 Core Equation

This appendix derives the hierarchical-shrinkage estimator of Agarwal et al. (2022) from its recursive definition, so that the single line implemented in src/papers/hierarchical_shrinkage.py can be checked by hand.

Setup. Let a fitted decision tree assign the input x a root-to-leaf path t_0, t_1, ..., t_L, where t_0 is the root and $t_L$the leaf. For a node t write N(t) for the number of training samples falling in it and mu(t) for the mean response of those samples, which for a classifier is the vector of class frequencies. The unshrunk tree predicts f(x) = mu(t_L).

Step 1: the telescoping identity. The leaf mean can be written as the root mean plus the sum of the increments along the path, since every intermediate term cancels:

$$\mu(t_L) = \mu(t_0) + \sum_{l=1}^{L} [\mu(t_l) - \mu(t_{l-1})]$$

This is exact and holds for any path. It re-expresses the prediction as a sequence of corrections, each attributable to one split, rather than as a single leaf value.

Step 2: the shrinkage recursion. Hierarchical shrinkage defines a shrunk value at each node by damping the increment inherited from its parent:

$$f_{HS}(t_0) = \mu(t_0), \quad f_{HS}(t_l) = f_{HS}(t_{l-1}) + [\mu(t_l) - \mu(t_{l-1})] \cdot w(t_{l-1})$$

$$w(t) = \frac{1}{1 + \lambda / N(t)}$$

The weight depends on the parent's sample count, not the child's. This is the design choice that makes the method behave sensibly: an increment estimated from a split of a large node is trusted, while an increment estimated from a split of an already-small node is discounted.

Step 3: unrolling the recursion. Expanding $f_H S(t_L)$ by repeated substitution of the previous line gives

$$f_{HS}(t_L) = \mu(t_0) + \sum_{l=1}^{L} \frac{\mu(t_l) - \mu(t_{l-1})}{1 + \lambda / N(t_{l-1})}$$

which is the estimator as published. Comparing it with Step 1 makes the mechanism transparent: hierarchical shrinkage is the telescoping decomposition of the tree with each term multiplied by a factor in (0, 1].

Step 4: limiting cases. Two checks confirm the form is sensible, and both are asserted in the unit tests.

- At lambda = 0 the weight is 1 for every node, the sum collapses back to Step 1, and $f_H S(x) = mu(t_L)$.Shrinkage with lambda = 0 is exactly the unshrunk tree.
- As lambda tends to infinity every weight tends to 0 and $f_H S(x)$ tends to mu(t_0), the global mean. The estimator therefore interpolates continuously between the unregularised tree and the constant predictor, with lambda as the single knob.
- For fixed lambda the weight tends to 1 as N(t) grows, so shrinkage is asymptotically negligible at nodes with plentiful data and strongest exactly where the estimate is least reliable.

Step 5: worked numerical example. Consider a three-node tree with root N = 1000, mu = 0.50 and a child with N = 20, mu = 0.90, at lambda = 25. The weight is w = 1 / (1 + 25/1000) = 0.97561, so the shrunk child value is 0.50 + (0.90 - 0.50) * 0.97561 = 0.89024. Had the same child descended from a parent with N = 40 instead, the weight would be 1 / (1 + 25/40) = 0.61538 and the shrunk value 0.74615. The identical raw increment is damped nearly seven times more strongly when the parent was small, which is the entire content of the method.

Step 6: extension to an ensemble. For a forest the shrinkage is applied independently to each tree and the shrunk trees are averaged as before. Because only the node values change, the tree structure is preserved exactly, so a single fitted forest can be re-shrunk at every lambda in the selection grid without any refitting - which is what makes internal cross-validated selection of lambda affordable.

Appendix B. Reproducibility

B.1 Environment

Table 40. Resolved environment for the run behind every number in this report.

Component	Version / role
Python	3.14
pandas	2.3.3
scikit-learn	1.8.0
XGBoost	installed via requirements pin
LightGBM	installed via requirements pin
SHAP (TreeSHAP)	installed via requirements pin
imbalanced-learn	SMOTE, BorderlineSMOTE, ADASYN baselines only
plotly	HTML figure generation
matplotlib	static figure generation
FastAPI + uvicorn	prediction service and latency harness

B.2 Fixed configuration and seeds

Table 41. Fixed configuration and random seeds. Every constant is pinned in source rather than passed at the command line.

Setting	Value
Global random state	0
Seed-repetition study	seeds 0, 1, 2
Snapshot minutes	0, 5, 10, ..., 90 (19 per match)
Margin clip	[-5, +5], applied to label and prediction
Probability floor	0.005 per class, renormalised
Class order	H, D, A (ordinal, used by RPS)
Train seasons	through 2020/2021
Validation seasons	2021/2022, 2022/2023
Test seasons	2023/2024, 2024/2025
Out-of-scope seasons	2025/2026
Tuning candidates / folds	12 per model-task, 3 folds
Tuning subsample cap	8,000 rows
Bootstrap resamples	2,000, clustered on match id
Multiple-comparison correction	Holm-Bonferroni within task, alpha = 0.05
Rating parameters	Elo K = 10, home advantage = 90, pi lambda = 0.06
P2 lambda grid	0, 0.1, 1, 10, 25, 50, 100

B.3 Pipeline stages and how to reproduce

The full pipeline is fifty-one stages and runs end to end with a single command from the repository root. The run behind this report executed on 30 August 2026 from 09:43 to 17:09, a wall clock of 7.4 hours, with every stage exiting zero.

```
python src/pipeline/download_extended_data.py

python run_pipeline.py
```

The stages proceed in five groups: (A) the data layer - download, store construction, team registry and alias map, cleaning, splits, odds baseline and rating layers; (B) the feature tables - pre-match, extended pre-match, stat-form, in-play snapshots; (C) hyperparameter tuning; (D) the model sweep, ensembles, final serving bundles and market comparison; and (E) the offline analyses - ablation, resampling, significance, SHAP, in-play curves, calibration, kernel scaling, compute profile and latency. Each stage writes its outputs to src/reports/ and each results file carries the input digests that produced it, so a stale intermediate cannot silently propagate.

B.4 Determinism check

Two independent determinism checks are reported. First, a season of new data was added to the store - 2,903 matches - while the four split constants remained pinned. Comparing all thirty-six model rows before and after, the maximum absolute change in RPS and in mean absolute error was **zero**, and train and test counts were unchanged on every row. That is the pinned-split contract of Section 2.6 doing exactly its job, and it is a stronger reproducibility claim than any statement about seeds. Second, the seed-repetition study refits every model under three seeds; the resulting spread is reported in Appendix C and is smaller than every difference this report calls significant.

Appendix C. Complete Result Tables

This appendix carries the complete versions of tables that appear abridged in the body, so that no computed value produced by the run is omitted.

C.1 Every pairwise comparison that survives Holm correction

__Table 42.__ All 60 comparisons surviving Holm-Bonferroni correction, including the trivial dummy comparisons. Excluding the dummy, 28 of 115 pairs are significant.

Task	Model A	Model B	n	Mean diff	CI low	CI high	Holm p	Verdict
C	dummy	gbm	5808	+0.03517	+0.03188	+0.03846	0.0000	gbm better
C	dummy	kernel_svm	5808	+0.02954	+0.02651	+0.03274	0.0000	kernel_svm better
C	dummy	lightgbm	5808	+0.03435	+0.03112	+0.03765	0.0000	lightgbm better
C	dummy	p2_hier_shrinkage	5808	+0.03547	+0.03213	+0.03885	0.0000	p2_hier_shrinkage better
C	dummy	random_forest	5808	+0.03527	+0.03191	+0.03864	0.0000	random_forest better
C	dummy	stack	5808	+0.03485	+0.03145	+0.03822	0.0000	stack better
C	dummy	stack_temporal	5808	+0.03545	+0.03206	+0.03880	0.0000	stack_temporal better
C	dummy	xgboost	5808	+0.03558	+0.03214	+0.03895	0.0000	xgboost better
C	gbm	kernel_svm	5808	-0.00563	-0.00703	-0.00418	0.0000	gbm better
C	kernel_svm	lightgbm	5808	+0.00481	+0.00344	+0.00619	0.0000	lightgbm better
C	kernel_svm	p2_hier_shrinkage	5808	+0.00593	+0.00445	+0.00735	0.0000	p2_hier_shrinkage better
C	kernel_svm	random_forest	5808	+0.00573	+0.00421	+0.00721	0.0000	random_forest better
C	kernel_svm	stack	5808	+0.00531	+0.00389	+0.00675	0.0000	stack better
C	kernel_svm	stack_temporal	5808	+0.00591	+0.00447	+0.00734	0.0000	stack_temporal better
C	kernel_svm	xgboost	5808	+0.00604	+0.00453	+0.00751	0.0000	xgboost better
C	lightgbm	stack_temporal	5808	+0.00110	+0.00046	+0.00172	0.0105	stack_temporal better
C	lightgbm	xgboost	5808	+0.00123	+0.00055	+0.00190	0.0200	xgboost better
C	stack	stack_temporal	5808	+0.00060	+0.00019	+0.00101	0.0475	stack_temporal better
R	dummy	gbm	5808	+0.18676	+0.16892	+0.20544	0.0000	gbm better
R	dummy	kernel_ridge_exact	5808	+0.16770	+0.15209	+0.18328	0.0000	kernel_ridge_exact better
R	dummy	kernel_ridge_nystroem	5808	+0.17605	+0.15957	+0.19323	0.0000	kernel_ridge_nystroem better
R	dummy	kernel_svr	5808	+0.17424	+0.15894	+0.19013	0.0000	kernel_svr better
R	dummy	lightgbm	5808	+0.18624	+0.16834	+0.20519	0.0000	lightgbm better
R	dummy	p2_hier_shrinkage	5808	+0.18427	+0.16642	+0.20295	0.0000	p2_hier_shrinkage better
R	dummy	random_forest	5808	+0.18441	+0.16627	+0.20317	0.0000	random_forest better
R	dummy	stack	5808	+0.18572	+0.16736	+0.20494	0.0000	stack better
R	dummy	stack_temporal	5808	+0.18588	+0.16712	+0.20564	0.0000	stack_temporal better
R	dummy	xgboost	5808	+0.18676	+0.16897	+0.20537	0.0000	xgboost better
R	gbm	kernel_ridge_exact	5808	-0.01906	-0.02628	-0.01153	0.0000	gbm better

Task	Model A	Model B	n	Mean diff	CI low	CI high	Holm p	Verdict
R	gbm	kernel_ridge_nystroem	5808	-0.01070	-0.01673	-0.00456	0.0360	gbm better
R	gbm	kernel_svr	5808	-0.01252	-0.01911	-0.00579	0.0360	gbm better
R	kernel_ridge_exact	kernel_ridge_nystroem	5808	+0.00835	+0.00405	+0.01312	0.0185	kernel_ridge_nystroem better
R	kernel_ridge_exact	lightgbm	5808	+0.01854	+0.01095	+0.02628	0.0000	lightgbm better
R	kernel_ridge_exact	p2_hier_shrinkage	5808	+0.01657	+0.00825	+0.02482	0.0000	p2_hier_shrinkage better
R	kernel_ridge_exact	random_forest	5808	+0.01670	+0.00888	+0.02469	0.0000	random_forest better
R	kernel_ridge_exact	stack	5808	+0.01802	+0.01027	+0.02607	0.0000	stack better
R	kernel_ridge_exact	stack_temporal	5808	+0.01818	+0.00989	+0.02652	0.0000	stack_temporal better
R	kernel_ridge_exact	xgboost	5808	+0.01906	+0.01148	+0.02651	0.0000	xgboost better
R	kernel_ridge_nystroem	lightgbm	5808	+0.01018	+0.00411	+0.01597	0.0360	lightgbm better
R	kernel_ridge_nystroem	xgboost	5808	+0.01071	+0.00476	+0.01653	0.0000	xgboost better
R	kernel_svr	lightgbm	5808	+0.01200	+0.00548	+0.01848	0.0360	lightgbm better
R	kernel_svr	stack	5808	+0.01148	+0.00460	+0.01834	0.0465	stack better
R	kernel_svr	xgboost	5808	+0.01252	+0.00578	+0.01906	0.0360	xgboost better
Lc	dummy	gbm	344	+0.07267	+0.05863	+0.08624	0.0000	gbm better
Lc	dummy	lightgbm	344	+0.07246	+0.05740	+0.08643	0.0000	lightgbm better
Lc	dummy	p2_hier_shrinkage	344	+0.07971	+0.06489	+0.09443	0.0000	p2_hier_shrinkage better
Lc	dummy	random_forest	344	+0.08253	+0.06892	+0.09590	0.0000	random_forest better
Lc	dummy	stack	344	+0.08125	+0.06771	+0.09500	0.0000	stack better
Lc	dummy	stack_temporal	344	+0.07867	+0.06425	+0.09318	0.0000	stack_temporal better
Lc	dummy	xgboost	344	+0.08019	+0.06616	+0.09378	0.0000	xgboost better
Lc	lightgbm	random_forest	344	+0.01007	+0.00361	+0.01641	0.0420	random_forest better
Lr	dummy	gbm	344	+0.50177	+0.40809	+0.59552	0.0000	gbm better
Lr	dummy	lightgbm	344	+0.49552	+0.40544	+0.58925	0.0000	lightgbm better
Lr	dummy	p2_hier_shrinkage	344	+0.51240	+0.41477	+0.61222	0.0000	p2_hier_shrinkage better
Lr	dummy	random_forest	344	+0.52982	+0.43841	+0.62331	0.0000	random_forest better
Lr	dummy	stack	344	+0.53364	+0.44117	+0.62915	0.0000	stack better
Lr	dummy	stack_temporal	344	+0.52342	+0.43235	+0.61823	0.0000	stack_temporal better
Lr	dummy	xgboost	344	+0.50562	+0.41696	+0.59669	0.0000	xgboost better
Lr	lightgbm	stack	344	+0.03812	+0.01737	+0.05795	0.0000	stack better
Lr	stack	xgboost	344	-0.02802	-0.04488	-0.01232	0.0300	stack better

C.2 Test metric across random seeds

Table 43. *Test metric under three random seeds, reported as a stability diagnostic. The spread column is the seed-to-seed range and is smaller than every difference this report calls significant.*

Task	Metric	Model	Seed 0	Seed 1	Seed 2	Spread
C	RPS	dummy	0.23039	0.23039	0.23039	0.00000
C	RPS	gbm	0.19470	0.19447	0.19460	0.00024
C	RPS	kernel_svm	0.20239	0.20240	0.20240	0.00001
C	RPS	lightgbm	0.19537	0.19532	0.19505	0.00032
C	RPS	p2_hier_shrinkage	0.19488	0.19511	0.19485	0.00026
C	RPS	random_forest	0.19477	0.19493	0.19488	0.00016
C	RPS	xgboost	0.19421	0.19407	0.19417	0.00014
Lc	RPS	dummy	0.22817	0.22817	0.22817	0.00000
Lc	RPS	gbm	0.16679	0.17054	0.17046	0.00374
Lc	RPS	lightgbm	0.15300	0.15438	0.15369	0.00138
Lc	RPS	p2_hier_shrinkage	0.15212	0.15396	0.15352	0.00184
Lc	RPS	random_forest	0.14319	0.14312	0.14260	0.00058
Lc	RPS	xgboost	0.14599	0.14583	0.14613	0.00029
Lr	MAE	dummy	1.48997	1.48997	1.48997	0.00000
Lr	MAE	gbm	0.98820	0.99118	0.99900	0.01080
Lr	MAE	lightgbm	0.99445	0.99748	1.00642	0.01197
Lr	MAE	p2_hier_shrinkage	0.97757	0.97640	0.97989	0.00349
Lr	MAE	random_forest	0.96015	0.95634	0.95991	0.00381
Lr	MAE	xgboost	0.98435	0.98129	0.98346	0.00306
R	MAE	dummy	1.43711	1.43711	1.43711	0.00000
R	MAE	gbm	1.25035	1.25075	1.25185	0.00150
R	MAE	kernel_ridge_exact	1.26941	1.27057	1.26830	0.00226
R	MAE	kernel_ridge_nystroem	1.26106	1.25956	1.25990	0.00150
R	MAE	kernel_svr	1.26287	1.26287	1.26287	0.00000
R	MAE	lightgbm	1.25087	1.24976	1.25024	0.00112
R	MAE	p2_hier_shrinkage	1.25284	1.25336	1.25202	0.00134
R	MAE	random_forest	1.25270	1.25264	1.25253	0.00018
R	MAE	xgboost	1.25035	1.24966	1.25079	0.00113

C.3 Per-model, per-phase calibration for Model 3

Table 44. *Complete per-model, per-phase calibration for Task Lc, all eight learners.*

Model	Phase	n	Mean conf.	Accuracy	Over-conf.	ECE	RPS	Log-loss	Brier
dummy	0-15	1032	0.4332	0.4855	-0.0522	0.05225	0.22812	1.05455	0.63585
dummy	15-30	1032	0.4332	0.4855	-0.0522	0.05225	0.22812	1.05455	0.63585
dummy	30-45	1032	0.4332	0.4855	-0.0522	0.05225	0.22812	1.05455	0.63585
dummy	45-60	1032	0.4332	0.4855	-0.0522	0.05225	0.22812	1.05455	0.63585
dummy	60-75	1032	0.4332	0.4855	-0.0522	0.05225	0.22812	1.05455	0.63585
dummy	75-90	1376	0.4332	0.4855	-0.0522	0.05225	0.22812	1.05455	0.63585
gbm	0-15	1032	0.5839	0.4797	+0.1042	0.11399	0.21313	1.06429	0.62349
gbm	15-30	1032	0.6001	0.5184	+0.0817	0.09262	0.20219	1.02190	0.59718
gbm	30-45	1032	0.6165	0.5417	+0.0749	0.08314	0.18513	0.95890	0.56450

Model	Phase	n	Mean conf.	Accuracy	Over-conf.	ECE	RPS	Log-loss	Brier
gbm	45-60	1032	0.6454	0.6047	+0.0408	0.07426	0.15220	0.84645	0.49106
gbm	60-75	1032	0.6706	0.6754	-0.0048	0.06530	0.12471	0.73485	0.42271
gbm	75-90	1376	0.7003	0.8118	-0.1114	0.11204	0.08039	0.53303	0.29601
lightgbm	0-15	1032	0.5960	0.4990	+0.0970	0.09973	0.22276	1.10704	0.64601
lightgbm	15-30	1032	0.6145	0.5233	+0.0913	0.09503	0.21152	1.06018	0.61882
lightgbm	30-45	1032	0.6348	0.5804	+0.0544	0.05581	0.18499	0.97783	0.56396
lightgbm	45-60	1032	0.6714	0.6453	+0.0260	0.04521	0.14738	0.83070	0.47790
lightgbm	60-75	1032	0.6990	0.7045	-0.0054	0.03390	0.11946	0.70946	0.40488
lightgbm	75-90	1376	0.7289	0.8176	-0.0887	0.11131	0.07480	0.48900	0.26944
p2_hier_shrinkage	0-15	1032	0.5829	0.4835	+0.0994	0.09938	0.21191	1.05094	0.61917
p2_hier_shrinkage	15-30	1032	0.6288	0.5194	+0.1094	0.10938	0.20186	1.02422	0.59633
p2_hier_shrinkage	30-45	1032	0.6563	0.5678	+0.0885	0.08846	0.18082	0.96020	0.54958
p2_hier_shrinkage	45-60	1032	0.7079	0.6453	+0.0625	0.06935	0.14579	0.82975	0.47531
p2_hier_shrinkage	60-75	1032	0.7502	0.7316	+0.0186	0.03288	0.11387	0.69451	0.38816
p2_hier_shrinkage	75-90	1376	0.7722	0.8721	-0.0999	0.10080	0.06427	0.44027	0.22912
random_forest	0-15	1032	0.5696	0.4758	+0.0939	0.09516	0.21402	1.03509	0.61743
random_forest	15-30	1032	0.6133	0.5155	+0.0978	0.10412	0.20317	1.00160	0.59207
random_forest	30-45	1032	0.6436	0.5678	+0.0758	0.07576	0.18027	0.94216	0.54488
random_forest	45-60	1032	0.7093	0.6453	+0.0639	0.06500	0.14320	0.81648	0.46538
random_forest	60-75	1032	0.7795	0.7355	+0.0441	0.05437	0.11181	0.67306	0.38416
random_forest	75-90	1376	0.8446	0.8910	-0.0464	0.05748	0.05219	0.37435	0.18972
stack	0-15	1032	0.5702	0.4642	+0.1060	0.10604	0.21740	1.05846	0.62936
stack	15-30	1032	0.6150	0.5116	+0.1034	0.10341	0.20585	1.03105	0.60258
stack	30-45	1032	0.6446	0.5746	+0.0700	0.07003	0.18089	0.95031	0.54639
stack	45-60	1032	0.7038	0.6492	+0.0545	0.05454	0.14311	0.80426	0.46313
stack	60-75	1032	0.7692	0.7316	+0.0376	0.04518	0.11148	0.67298	0.37965
stack	75-90	1376	0.8325	0.8903	-0.0578	0.05776	0.05361	0.38121	0.19315
stack_temporal	0-15	1032	0.5867	0.4767	+0.1100	0.11850	0.21724	1.06519	0.63126
stack_temporal	15-30	1032	0.6365	0.5213	+0.1152	0.12133	0.20687	1.03253	0.60550
stack_temporal	30-45	1032	0.6677	0.5533	+0.1144	0.11479	0.18588	0.97483	0.56387
stack_temporal	45-60	1032	0.7219	0.6250	+0.0969	0.09691	0.14870	0.83451	0.48159
stack_temporal	60-75	1032	0.7776	0.7209	+0.0566	0.05713	0.11454	0.69310	0.39163
stack_temporal	75-90	1376	0.8266	0.8852	-0.0585	0.05860	0.05496	0.38769	0.19903
xgboost	0-15	1032	0.5636	0.4632	+0.1004	0.10106	0.22001	1.06855	0.63945
xgboost	15-30	1032	0.6057	0.5078	+0.0979	0.09790	0.20613	1.01994	0.60401
xgboost	30-45	1032	0.6430	0.5611	+0.0820	0.08199	0.18112	0.94208	0.55065
xgboost	45-60	1032	0.7084	0.6444	+0.0640	0.06403	0.14304	0.80231	0.46669
xgboost	60-75	1032	0.7680	0.7229	+0.0451	0.04514	0.11048	0.66554	0.38120
xgboost	75-90	1376	0.8157	0.8772	-0.0615	0.06924	0.05708	0.39400	0.20891

C.4 The forty worst predictions in full

Table 45. *The forty worst predictions, ten per task, with their adjudication.*

Task	Rank	Match id	Min	Loss	Actual	P(actual)	Pred margin	Market P	Verdict
C	1	2000024703	-	0.7864	A	0.0622	-	0.0632	inherently uncertain
C	2	2000002300	-	0.7832	A	0.0737	-	0.0451	inherently uncertain
C	3	2000019672	-	0.7806	A	0.0711	-	0.0551	inherently uncertain
C	4	2000016303	-	0.7784	A	0.0730	-	0.0797	inherently uncertain
C	5	2000024574	-	0.7729	A	0.0672	-	0.0938	inherently uncertain
C	6	2000024922	-	0.7684	A	0.0740	-	0.0624	inherently uncertain

Task	Rank	Match id	Min	Loss	Actual	P(actual)	Pred margin	Market P	Verdict
C	7	2000009225	-	0.7664	A	0.0664	-	0.0733	inherently uncertain
C	8	2000002667	-	0.7661	A	0.0747	-	0.0497	inherently uncertain
C	9	2000009252	-	0.7658	A	0.0712	-	0.0495	inherently uncertain
C	10	2000024932	-	0.7583	A	0.0798	-	0.0550	inherently uncertain
R	1	2000005375	-	6.4540	5.0	-	-1.4540	-	inherently uncertain
R	2	2000005240	-	5.5185	-5.0	-	0.5185	-	inherently uncertain
R	3	2000005413	-	5.4472	-5.0	-	0.4472	-	inherently uncertain
R	4	2000005244	-	5.4391	-5.0	-	0.4391	-	inherently uncertain
R	5	2000016479	-	5.4115	-4.0	-	1.4115	-	inherently uncertain
R	6	2000022662	-	5.3145	5.0	-	-0.3145	-	inherently uncertain
R	7	2000005561	-	5.2488	5.0	-	-0.2487	-	inherently uncertain
R	8	2000019476	-	5.1631	5.0	-	-0.1631	-	inherently uncertain
R	9	2000002260	-	5.1567	4.0	-	-1.1567	-	inherently uncertain
R	10	2000002166	-	5.0381	5.0	-	-0.0381	-	inherently uncertain
Lc	1	3754064	45	0.9258	H	0.0097	-	0.3722	noisy observation
Lc	2	3754064	40	0.9176	H	0.0133	-	0.3722	noisy observation
Lc	3	3754064	35	0.9105	H	0.0138	-	0.3722	noisy observation
Lc	4	3754064	30	0.9025	H	0.0138	-	0.3722	noisy observation
Lc	5	3754064	55	0.9003	H	0.0176	-	0.3722	noisy observation
Lc	6	3754064	60	0.9002	H	0.0177	-	0.3722	noisy observation
Lc	7	3754064	50	0.8982	H	0.0180	-	0.3722	noisy observation
Lc	8	3754064	25	0.8847	H	0.0162	-	0.3722	noisy observation
Lc	9	3879846	70	0.8821	H	0.0192	-	0.1906	noisy observation
Lc	10	3754064	80	0.8812	H	0.0176	-	0.3722	noisy observation
Lr	1	3879847	0	4.8273	-4.0	-	0.8273	-	inherently uncertain
Lr	2	3879847	5	4.8124	-4.0	-	0.8124	-	inherently uncertain
Lr	3	3879847	10	4.7768	-4.0	-	0.7768	-	inherently uncertain
Lr	4	3754333	5	4.5316	4.0	-	-0.5316	-	inherently uncertain
Lr	5	3754333	0	4.4859	4.0	-	-0.4859	-	inherently uncertain
Lr	6	3879835	5	4.4662	4.0	-	-0.4662	-	noisy observation
Lr	7	3879835	10	4.4582	4.0	-	-0.4582	-	noisy observation
Lr	8	3754333	10	4.4483	4.0	-	-0.4484	-	inherently uncertain
Lr	9	3879835	30	4.4180	4.0	-	-0.4179	-	noisy observation
Lr	10	3879835	25	4.3982	4.0	-	-0.3982	-	noisy observation

C.5 Per-learner results for all six imbalance arms

Table 46. *Complete six-arm imbalance study, four learners per arm, test block.*

Arm	Learner	Train rows	RPS	RPS raw	Log-loss	Brier	ECE	Draw recall	Draw F1
vanilla	random_forest	39707	0.19481	0.19477	0.97195	0.57755	0.01303	0.0000	0.0000
vanilla	gbm	39707	0.19491	0.19471	0.97346	0.57817	0.01795	0.0000	0.0000
vanilla	kernel_svm	39707	0.20054	0.20239	0.99381	0.59179	0.02805	0.0000	0.0000
vanilla	lightgbm	39707	0.19573	0.19537	0.97714	0.58040	0.02745	0.0027	0.0053
smote	random_forest	54108	0.19528	0.19891	0.97340	0.57871	0.01888	0.0000	0.0000
smote	gbm	54108	0.19584	0.19561	0.97898	0.58119	0.02814	0.0034	0.0066
smote	kernel_svm	54108	0.20268	0.20359	1.00097	0.59635	0.02655	0.0081	0.0157
smote	lightgbm	54108	0.19519	0.19510	0.97541	0.57946	0.02652	0.0027	0.0053
borderline_smote	random_forest	54108	0.19564	0.19962	0.97467	0.57939	0.01774	0.0000	0.0000
borderline_smote	gbm	54108	0.19556	0.19580	0.97878	0.58086	0.02521	0.0007	0.0013

Arm	Learner	Train rows	RPS	RPS raw	Log-loss	Brier	ECE	Draw recall	Draw F1
borderline_smote	kernel_svm	54108	0.20324	0.20422	1.00249	0.59760	0.02312	0.0215	0.0402
borderline_smote	lightgbm	54108	0.19550	0.19564	0.97767	0.58063	0.02670	0.0000	0.0000
adasyn	random_forest	56390	0.19575	0.20181	0.97474	0.57969	0.01851	0.0000	0.0000
adasyn	gbm	56390	0.19655	0.19714	0.98055	0.58274	0.02288	0.0000	0.0000
adasyn	kernel_svm	56390	0.20522	0.20628	1.00958	0.60165	0.02203	0.0188	0.0357
adasyn	lightgbm	56390	0.19590	0.19621	0.97935	0.58123	0.02174	0.0020	0.0040
class_weight	random_forest	39707	0.19527	0.20002	0.97351	0.57870	0.01948	0.0000	0.0000
class_weight	gbm	39707	0.19475	0.20122	0.97348	0.57792	0.02430	0.0013	0.0027
class_weight	kernel_svm	39707	0.19962	0.19959	0.99335	0.58992	0.02433	0.0195	0.0364
class_weight	lightgbm	39707	0.19559	0.20225	0.97753	0.58043	0.02320	0.0013	0.0027
p1_gsmotenc	random_forest	54108	0.19530	0.19583	0.97309	0.57863	0.01808	0.0013	0.0027
p1_gsmotenc	gbm	54108	0.19477	0.19395	0.97482	0.57826	0.02308	0.0007	0.0013
p1_gsmotenc	kernel_svm	54108	0.20013	0.20655	0.99664	0.59184	0.02614	0.0242	0.0440
p1_gsmotenc	lightgbm	54108	0.19496	0.19433	0.97427	0.57877	0.02066	0.0027	0.0053

C.6 In-play feature-group ablation for both heads

Table 47. *Task Lc feature-group ablation in full, validation block, uncalibrated, random forest.*

Configuration	Columns	Validation RPS	Delta
prematch_only	240	0.19279	+0.06683
drop_inplay_score	388	0.17380	+0.04784
drop_pi_ratings	393	0.12700	+0.00104
drop_agg_possession	376	0.12697	+0.00101
drop_elo	392	0.12684	+0.00088
drop_inplay_tempo	386	0.12678	+0.00082
drop_inplay_shots	377	0.12664	+0.00068
drop_agg_scoring	388	0.12654	+0.00058
drop_inplay_passing	374	0.12653	+0.00057
drop_league	390	0.12652	+0.00056
drop_xi_ratings	393	0.12648	+0.00052
drop_form_results	388	0.12647	+0.00051
drop_agg_discipline	391	0.12646	+0.00050
drop_inplay_possession	384	0.12644	+0.00048
drop_squad_ratings	393	0.12644	+0.00048
drop_venue_form	374	0.12640	+0.00044
drop_agg_passing	370	0.12638	+0.00042
drop_inplay_defence	369	0.12636	+0.00040
drop_form_xg	388	0.12633	+0.00037
drop_agg_shooting	370	0.12621	+0.00025
drop_agg_defence	364	0.12615	+0.00019
drop_agg_set_pieces	361	0.12615	+0.00019
drop_schedule	389	0.12613	+0.00017
drop_form_goals	388	0.12613	+0.00017
drop_inplay_pressure	374	0.12606	+0.00010
drop_inplay_discipline	393	0.12601	+0.00005
drop_inplay_territory	369	0.12600	+0.00004
drop_head_to_head	383	0.12600	+0.00004

Configuration	Columns	Validation RPS	Delta
full	394	0.12596	+0.00000
drop_agg_pressure	370	0.12569	-0.00027
drop_agg_territory	364	0.12556	-0.00040
drop_inplay_xg	387	0.12539	-0.00057

Table 48. *Task Lr feature-group ablation in full, validation block, uncalibrated, random forest.*

Configuration	Columns	Validation MAE	Delta
prematch_only	240	1.17218	+0.33740
drop_inplay_score	388	1.07236	+0.23758
drop_inplay_xg	387	0.83592	+0.00114
drop_venue_form	374	0.83577	+0.00099
drop_agg_set_pieces	361	0.83510	+0.00032
drop_inplay_passing	374	0.83504	+0.00026
full	394	0.83478	+0.00000
drop_agg_defence	364	0.83456	-0.00022
drop_agg_shooting	370	0.83357	-0.00121
drop_agg_passing	370	0.83331	-0.00147
drop_agg_scoring	388	0.83269	-0.00209
drop_league	390	0.83266	-0.00212
drop_xi_ratings	393	0.83125	-0.00353
drop_inplay_tempo	386	0.83107	-0.00371
drop_inplay_territory	369	0.83103	-0.00375
drop_form_results	388	0.83071	-0.00407
drop_pi_ratings	393	0.83071	-0.00407
drop_squad_ratings	393	0.83052	-0.00426
drop_inplay_defence	369	0.82911	-0.00567
drop_elo	392	0.82904	-0.00574
drop_inplay_pressure	374	0.82898	-0.00580
drop_form_goals	388	0.82890	-0.00588
drop_agg_discipline	391	0.82875	-0.00603
drop_form_xg	388	0.82862	-0.00616
drop_head_to_head	383	0.82825	-0.00653
drop_agg_territory	364	0.82755	-0.00723
drop_inplay_discipline	393	0.82754	-0.00724
drop_agg_pressure	370	0.82696	-0.00782
drop_inplay_possession	384	0.82674	-0.00804
drop_inplay_shots	377	0.82441	-0.01037
drop_schedule	389	0.82346	-0.01132
drop_agg_possession	376	0.82223	-0.01255

C.7 Full hyperparameter search record

Table 49. *Hyperparameter search budget and best cross-validated score per model and task. 288 candidate evaluations were run in total.*

Task	Model	Candidates	Folds	Tuning rows	Subsampled	Objective	Best CV score
C	gbm	12	3	8000	yes	log_loss	0.99428
C	kernel_svm	12	3	8000	yes	log_loss	1.00619
C	lightgbm	12	3	8000	yes	log_loss	0.99527
C	p2_hier_shrinkage	12	3	8000	yes	log_loss	0.98289
C	random_forest	12	3	8000	yes	log_loss	0.98286
C	xgboost	12	3	8000	yes	log_loss	0.98393
Lc	gbm	12	3	7999	yes	log_loss	1.20104
Lc	lightgbm	12	3	7999	yes	log_loss	1.02306
Lc	p2_hier_shrinkage	12	3	7999	yes	log_loss	0.89608
Lc	random_forest	12	3	7999	yes	log_loss	0.84354
Lc	xgboost	12	3	7999	yes	log_loss	0.90157
Lr	gbm	12	3	7999	yes	mae	1.00570
Lr	lightgbm	12	3	7999	yes	mae	1.00687
Lr	p2_hier_shrinkage	12	3	7999	yes	mae	0.95917
Lr	random_forest	12	3	7999	yes	mae	0.93831
Lr	xgboost	12	3	7999	yes	mae	0.96943
R	gbm	12	3	8000	yes	mae	1.26375
R	kernel_ridge_exact	12	3	8000	yes	mae	1.26257
R	kernel_ridge_nystroem	12	3	8000	yes	mae	1.26321
R	kernel_svr	12	3	8000	yes	mae	1.26565
R	lightgbm	12	3	8000	yes	mae	1.26124
R	p2_hier_shrinkage	12	3	8000	yes	mae	1.25774
R	random_forest	12	3	8000	yes	mae	1.25738
R	xgboost	12	3	8000	yes	mae	1.25683

References

[1] Agarwal, A., Tan, Y. S., Ronen, O., Singh, C., & Yu, B. (2022). Hierarchical Shrinkage: Improving the accuracy and interpretability of tree-based models. Proceedings of the 39th International Conference on Machine Learning, PMLR 162, 111-135.

[2] Constantinou, A. C., & Fenton, N. E. (2012). Solving the problem of inadequate scoring rules for assessing probabilistic football forecast models. Journal of Quantitative Analysis in Sports, 8(1).

[3] Fonseca, J., & Bacao, F. (2023). Geometric SMOTE for imbalanced datasets with nominal and continuous features. Expert Systems with Applications, 234, 121053.

[4] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357.

[5] Constantinou, A. C., & Fenton, N. E. (2013). Determining the level of ability of football teams by dynamic ratings based on the relative discrepancies in scores between adversaries. Journal of Quantitative Analysis in Sports, 9(1), 37-50.

[6] Elo, A. E. (1978). The Rating of Chessplayers, Past and Present. Arco Publishing.

[7] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30.

[8] Lundberg, S. M., Erion, G., Chen, H., et al. (2020). From local explanations to global understanding with explainable AI for trees. Nature Machine Intelligence, 2, 56-67.

[9] Zadrozny, B., & Elkan, C. (2002). Transforming classifier scores into accurate multiclass probability estimates. Proceedings of KDD 2002, 694-699.

[10] Holm, S. (1979). A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics, 6(2), 65-70.

[11] Efron, B., & Tibshirani, R. J. (1993). An Introduction to the Bootstrap. Chapman & Hall.

[12] Williams, C. K. I., & Seeger, M. (2001). Using the Nystroem method to speed up kernel machines. Advances in Neural Information Processing Systems, 13, 682-688.

[13] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of KDD 2016, 785-794.

[14] Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems, 30.

[15] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

[16] Wolpert, D. H. (1992). Stacked generalization. Neural Networks, 5(2), 241-259.

[17] StatsBomb (2024). StatsBomb Open Data. https://github.com/statsbomb/open-data

[18] Football-Data.co.uk (2026). Historical results and betting odds. https://www.football-data.co.uk

[19] Mathien, H. (2016). European Soccer Database. Mirrored at https://huggingface.co/datasets/julien-c/kaggle-hugomathien-soccer

[20] sofifa player ratings archive, mirrored at https://huggingface.co/datasets/jsulz/FIFA23

List of Tables

Every entry links to the table it names. Page numbers refer to the printed pagination.

[Return to Contents]

List of Figures

Every entry links to the figure it names. Page numbers refer to the printed pagination.

[Return to Contents]